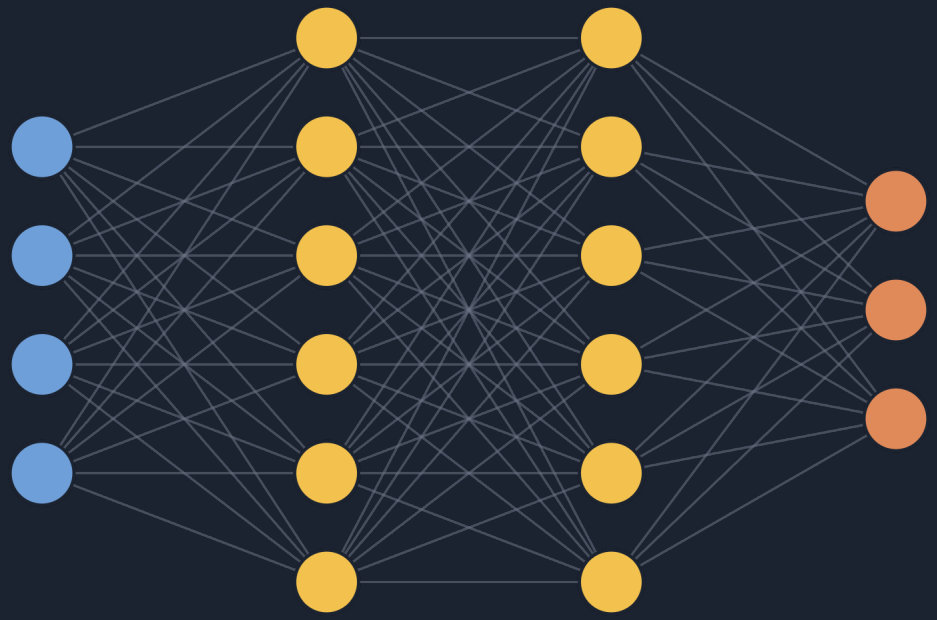
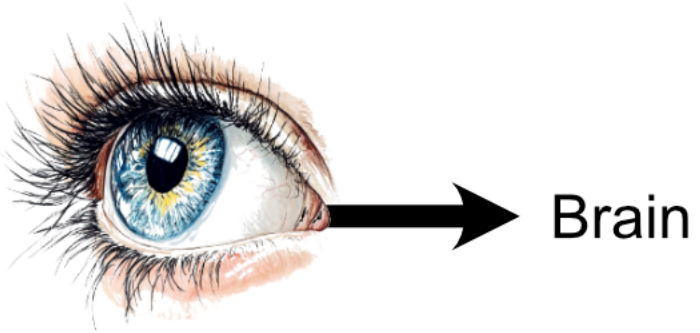
MACHINE LEARNING · LECTURE

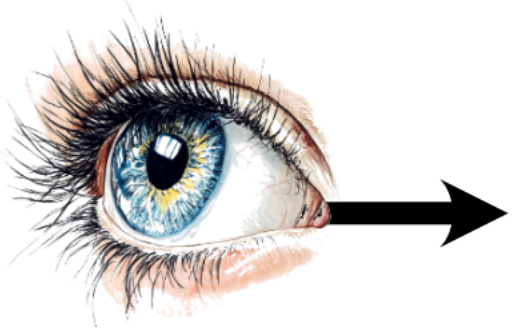
Neural networks

From one neuron to learning the features yourself





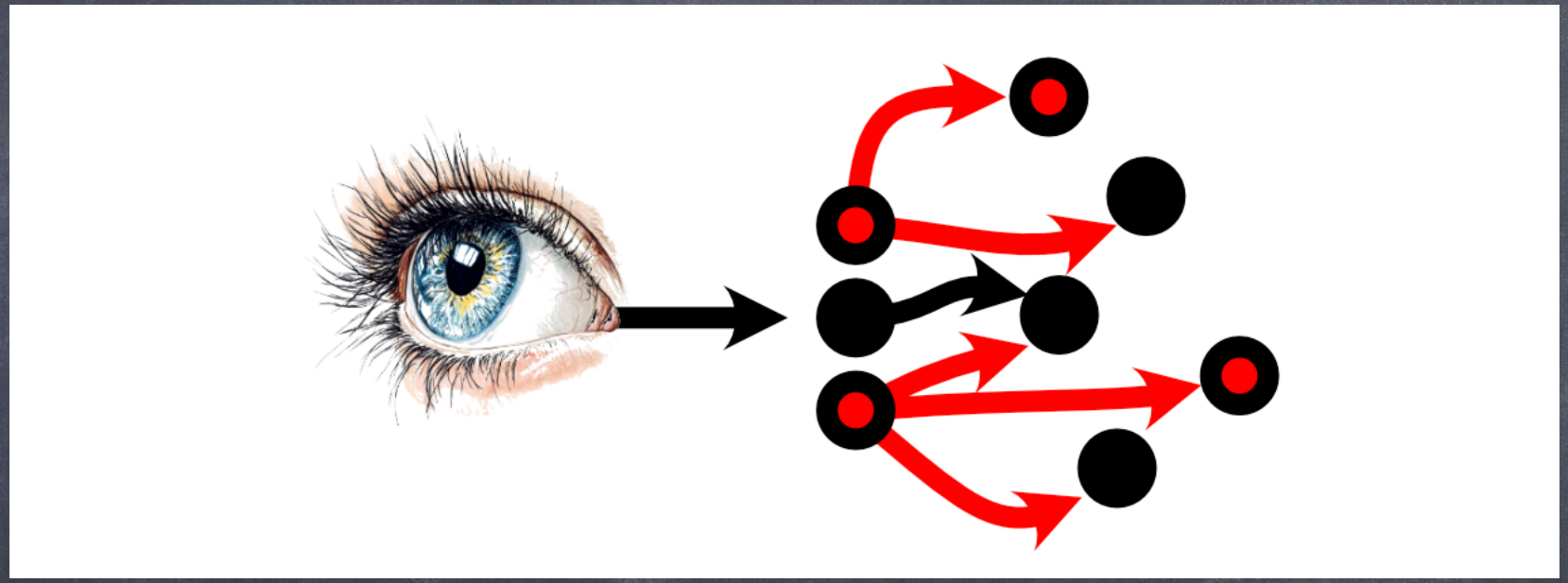
Brain

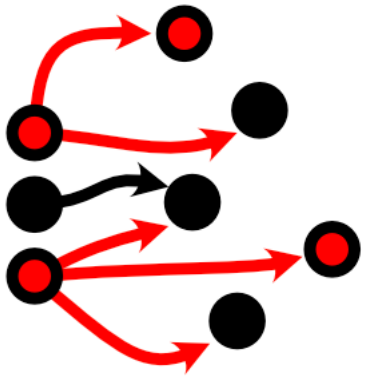


Fire

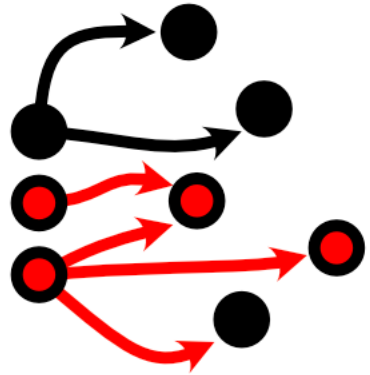
Don't Fire

Fire

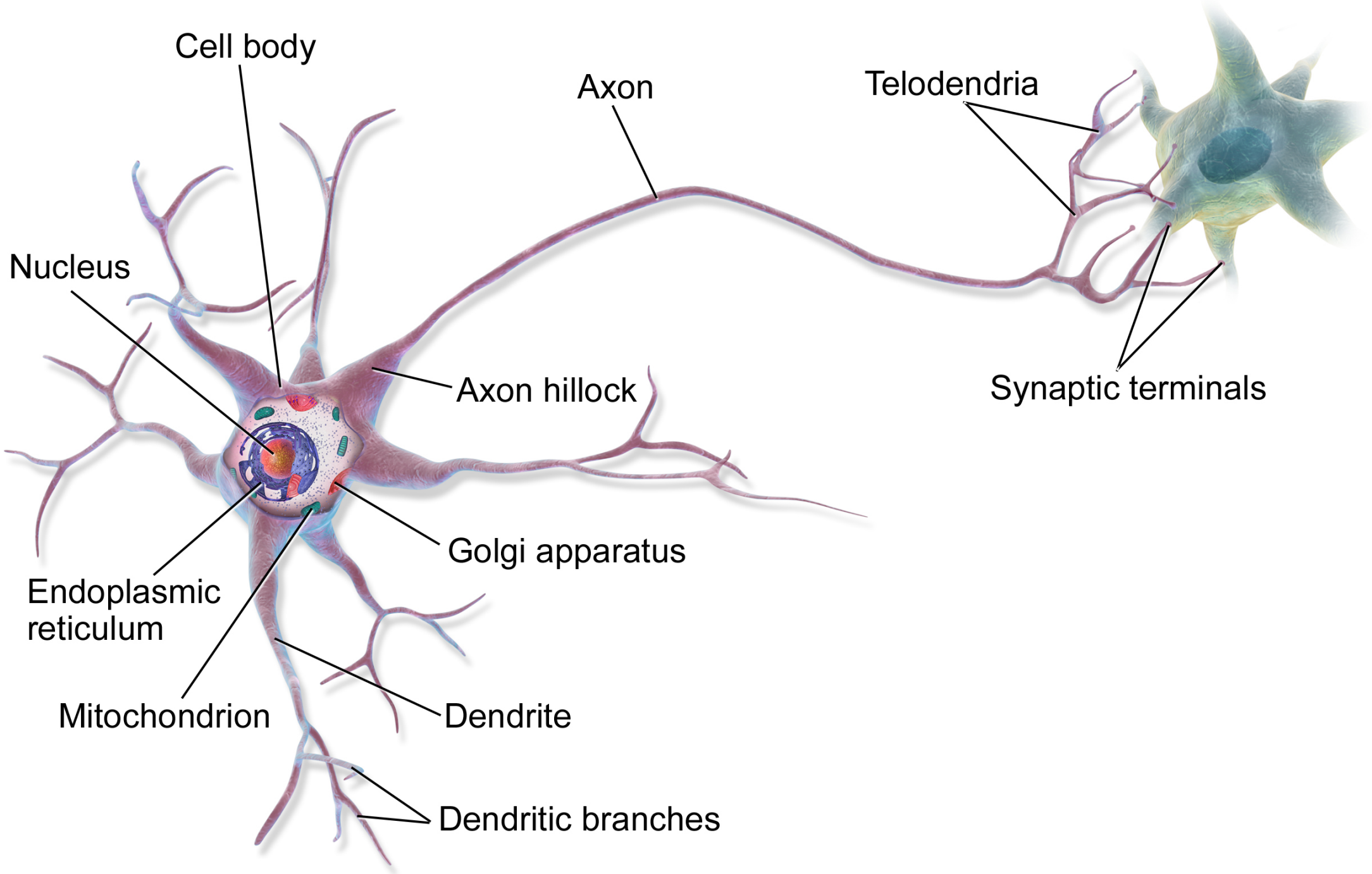




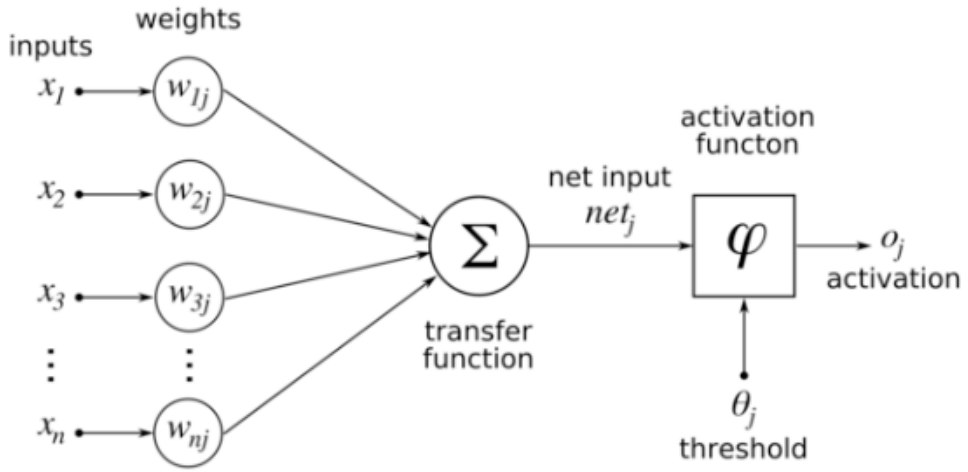




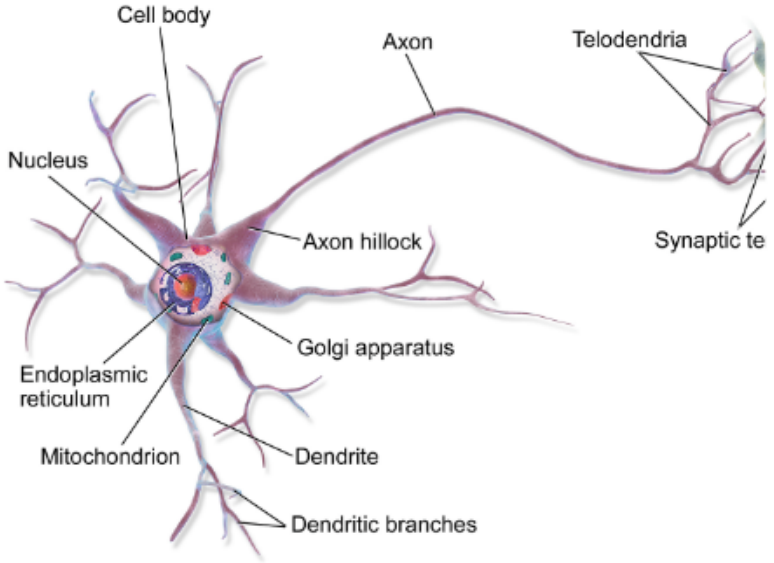



Cell body
Axon
Telodendria
Nucleus
Axon hillock
Synaptic terminals
Golgi apparatus
Endoplasmic reticulum
Mitochondrion
Dendrite
Dendritic branches

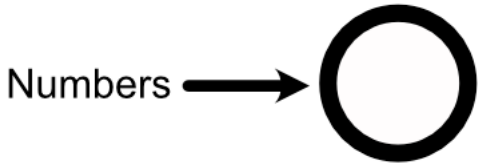
A Perceptron



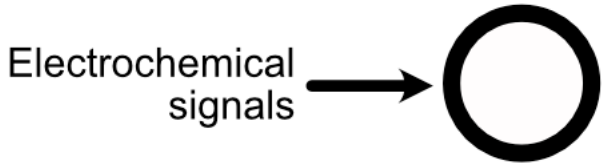
A Neuron



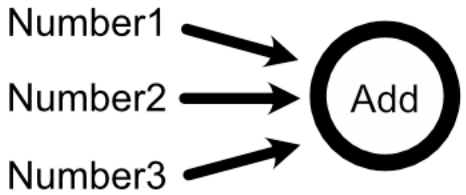
A Perceptron



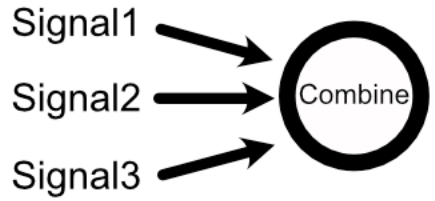
A Neuron



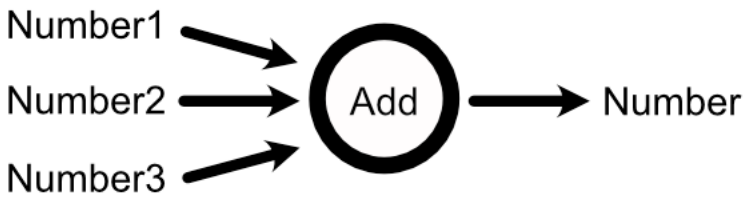
A Perceptron



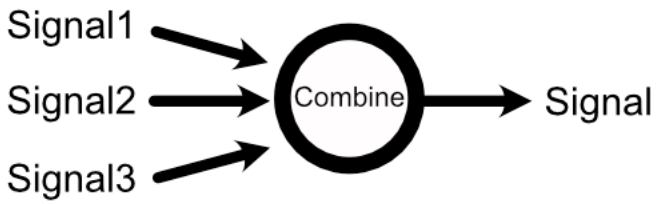
A Neuron



A Perceptron

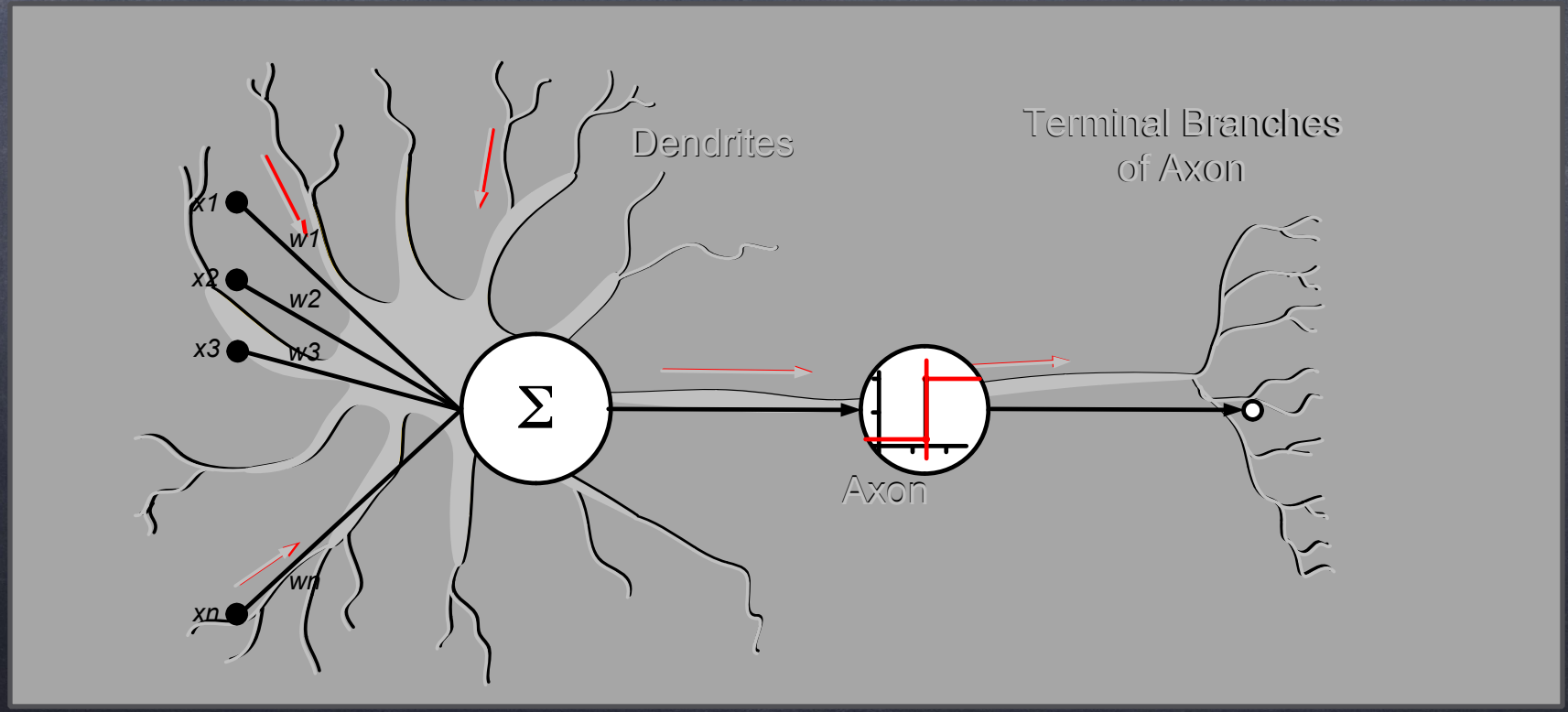


A Neuron

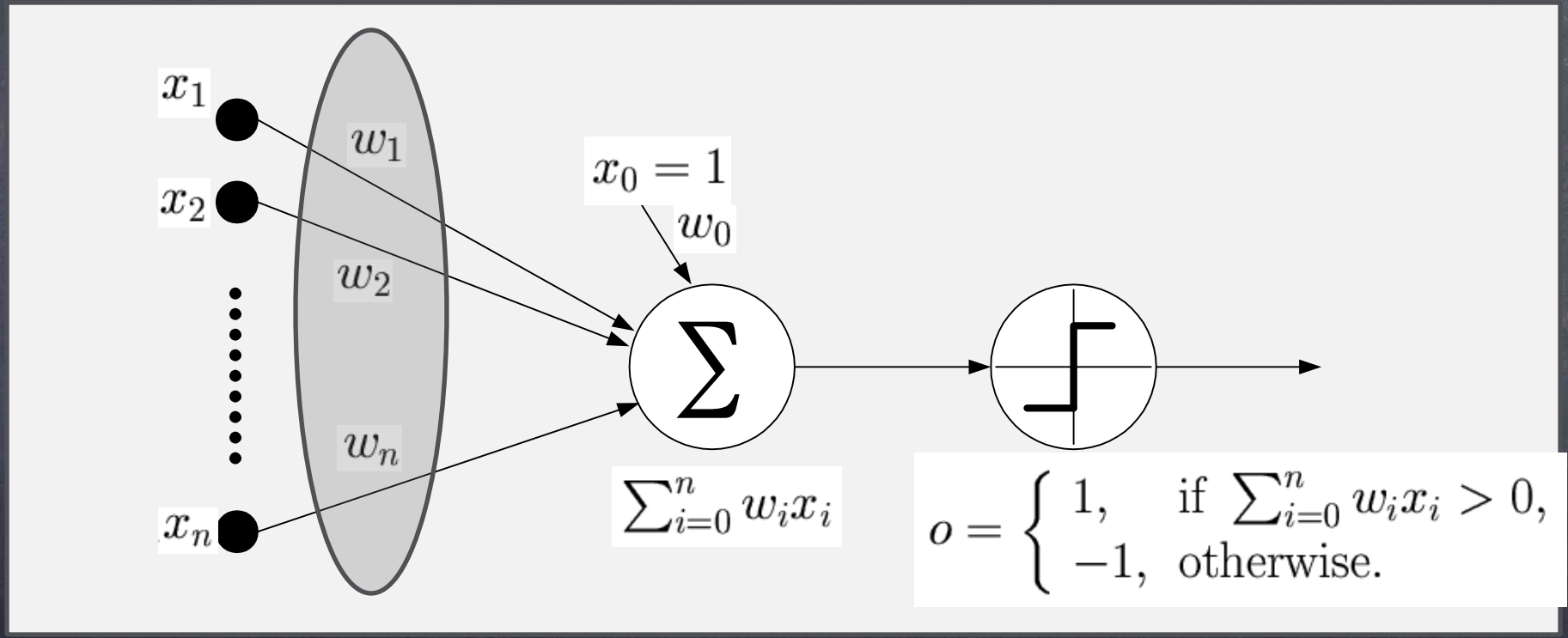


A Simple Artificial Neuron

- Perceptron



What is here to learn?

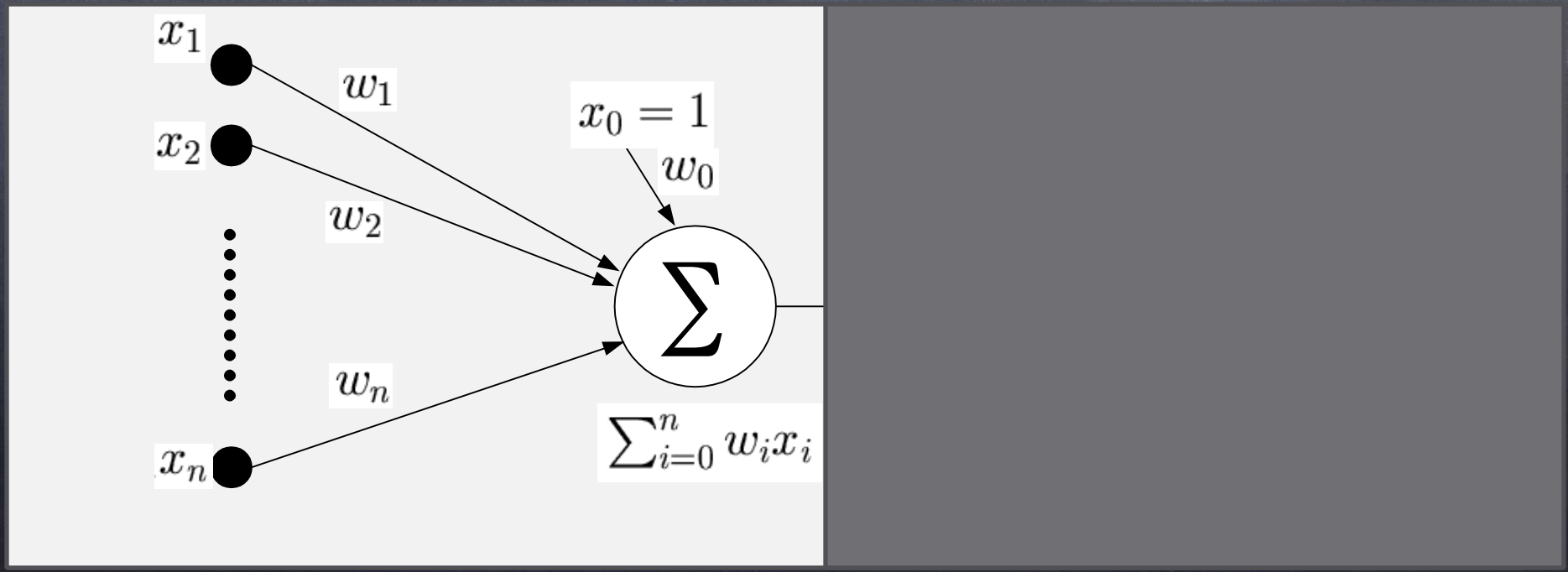


The Perceptron Training Rule

Gradient Descent and Delta Rule

- The delta training rule is best understood by considering the task of training an *unthresholded* perceptron; that is, a *linear unit* for which the output *o* is given by

$$o(\vec{x}) = \vec{w} \cdot \vec{x}$$



Gradient Descent and Delta Rule

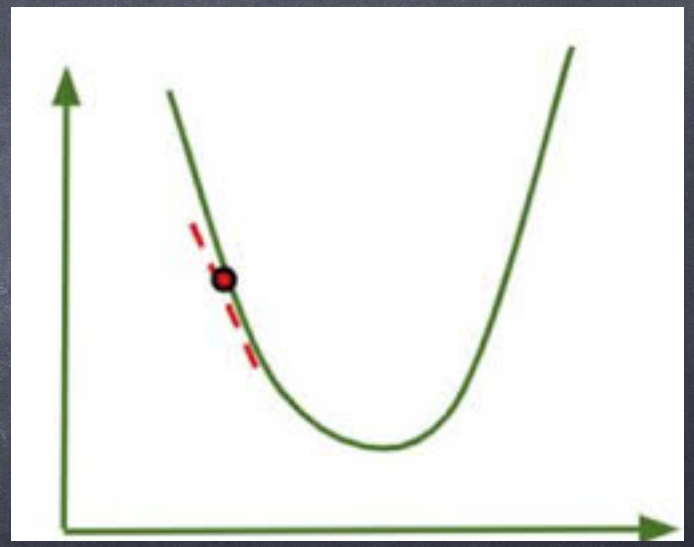
In order to derive a weight learning rule for linear units, let us begin by specifying a measure for the training error of a hypothesis (weight vector), relative to the training examples.

$$E(\vec{w}) \equiv \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2$$

Derivation GDR

The vector derivative is called the gradient of E $\vec{w}$ with respect to

$$\nabla E(\vec{w}) \equiv \left[\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \cdots, \frac{\partial E}{\partial w_n}\right]$$



The gradient specifies the direction that produces the steepest increase in E. The negative of this vector therefore gives the direction of steepest decrease. The training rule for gradient descent is

$$\vec{w} \leftarrow \vec{w} + \Delta\vec{w} \quad \text{where} \quad \Delta\vec{w} = -\eta\nabla E(\vec{w})$$

Derivation of GDR …

The vector of $\frac{\partial E}{\partial w_i}$ derivatives that form the gradient can be obtained by differentiating E

$$\begin{aligned}
\frac{\partial E}{\partial w_i} &= \frac{\partial}{\partial w_i}\frac{1}{2}\sum_{d\in D}(t_d - o_d)^2 \\
&= \frac{1}{2}\sum_{d\in D}\frac{\partial}{\partial w_i}(t_d - o_d)^2 \\
&= \frac{1}{2}\sum_{d\in D}2(t_d - o_d)\frac{\partial}{\partial w_i}(t_d - o_d) \\
&= \sum_{d\in D}(t_d - o_d)\frac{\partial}{\partial w_i}(t_d - \vec{w}\cdot\vec{x}_d) \\
\frac{\partial E}{\partial w_i} &= \sum_{d\in D}(t_d - o_d)(-x_{id})
\end{aligned}$$

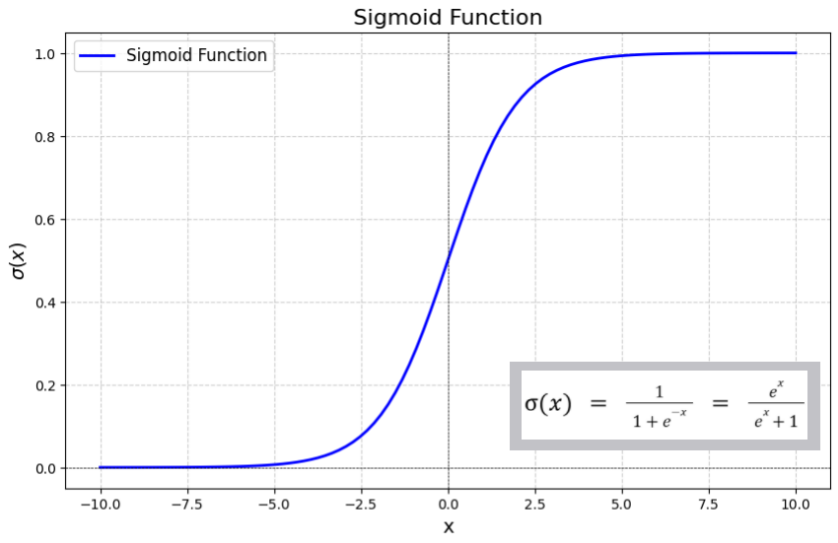
The weight update rule for standard gradient descent can be summarized as

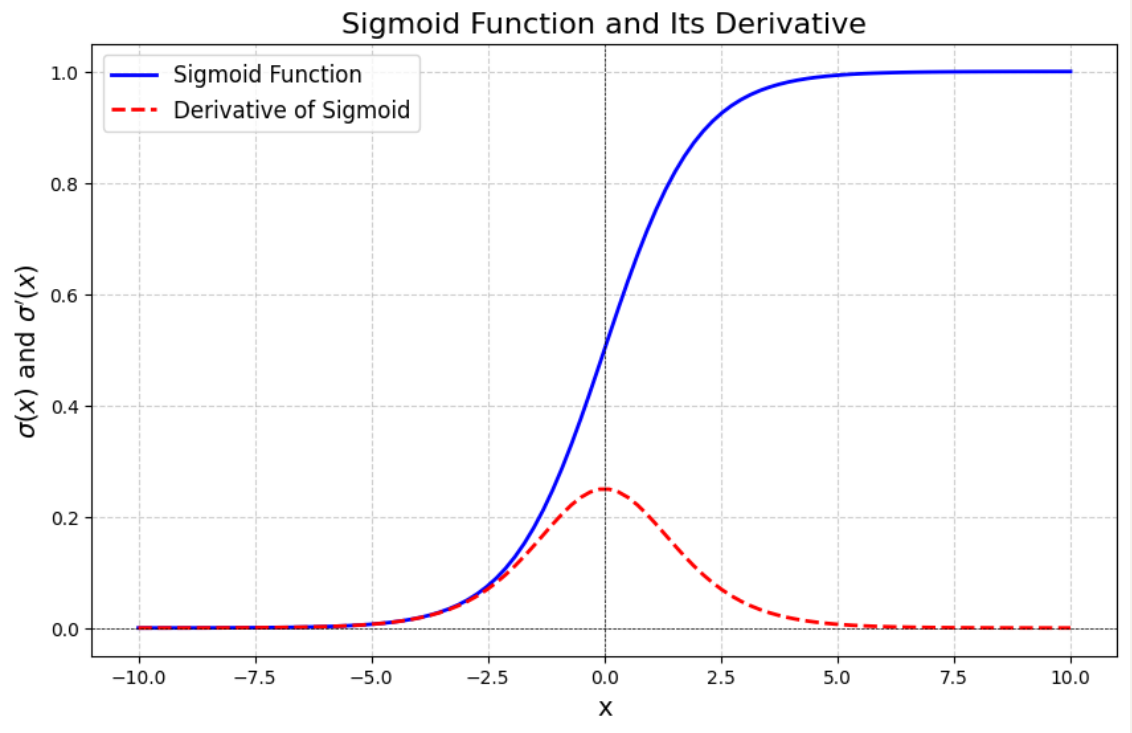
$$\Delta w_i = -\eta\frac{\partial E}{\partial w_i}$$

$$\Delta w_i = \eta\sum_{d\in D}(t_d - o_d)x_{id}$$

Activation functions

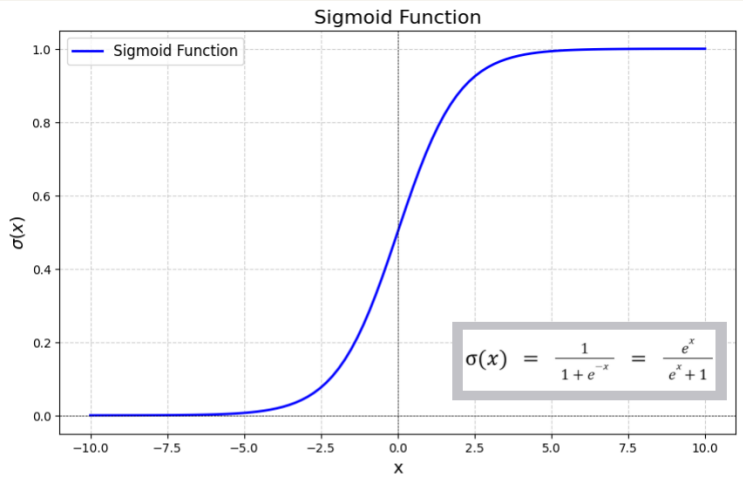
Sigmoid



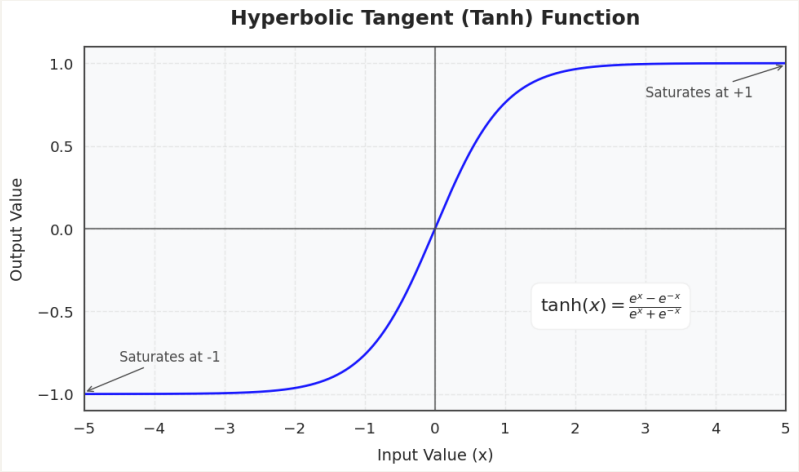


Activation functions

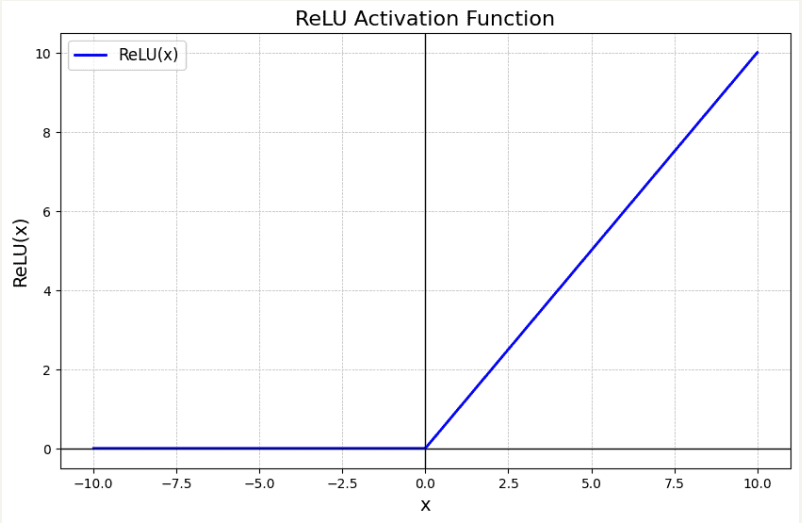
Sigmoid



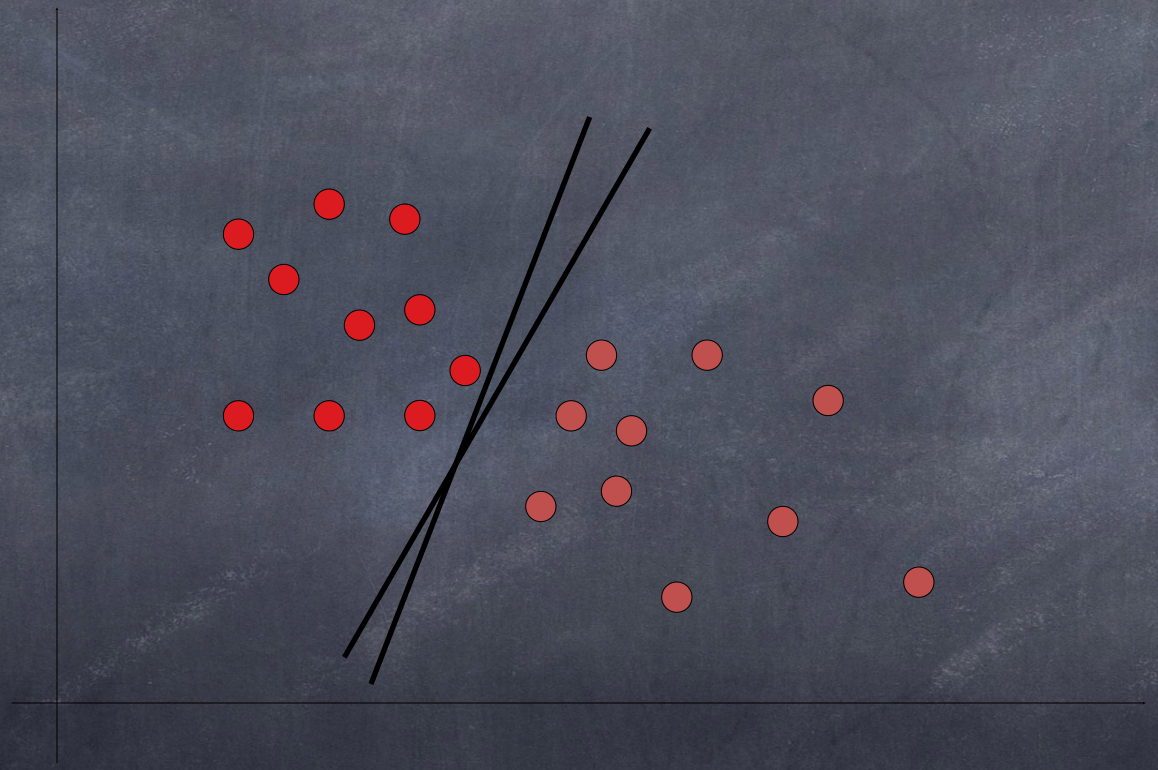
tanh



ReLU max(0, z)



AB problem



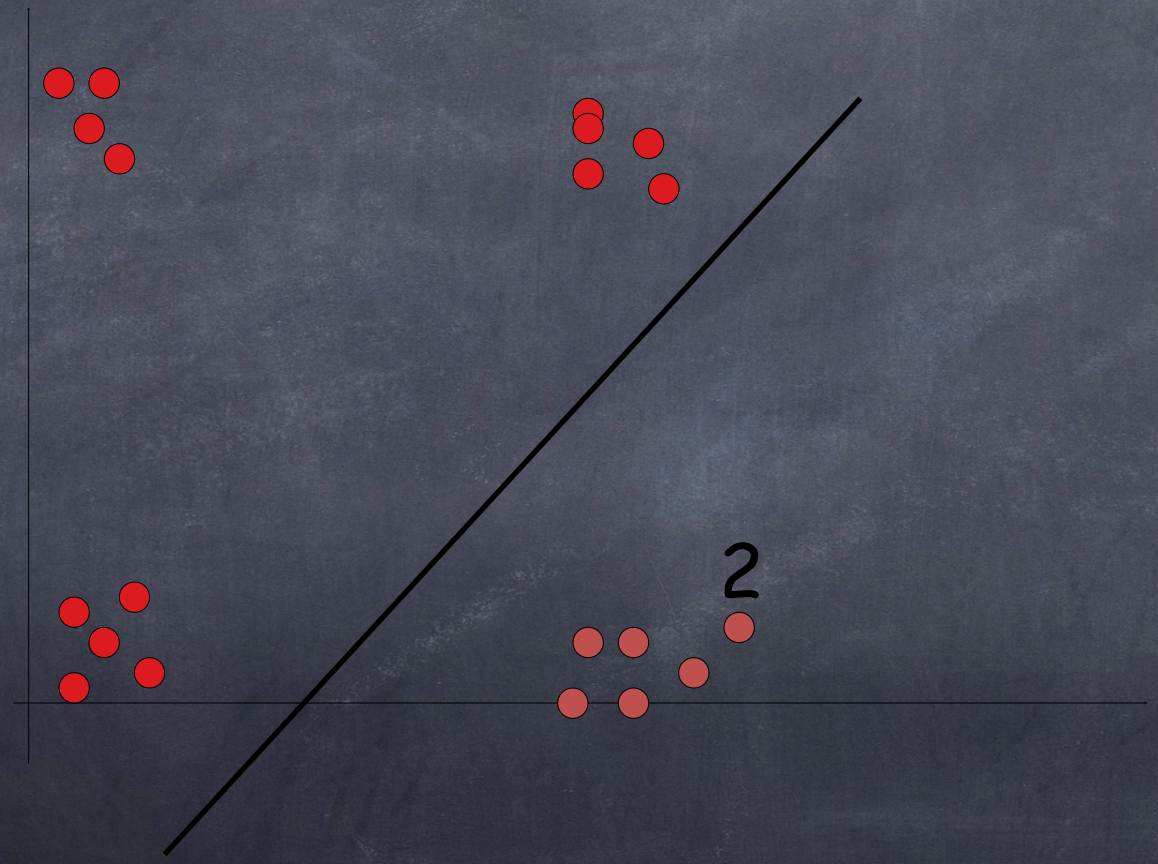
XOR problem



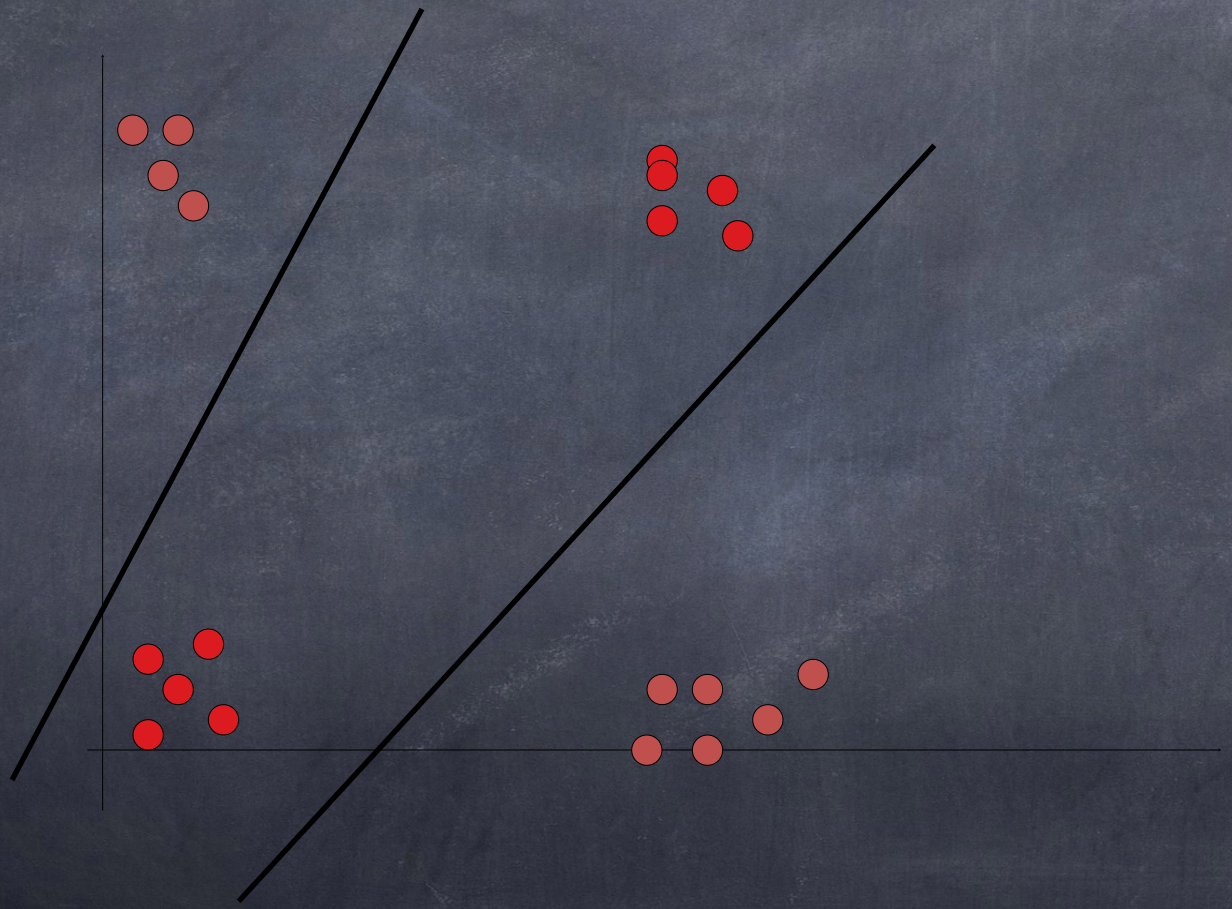
XOR problem



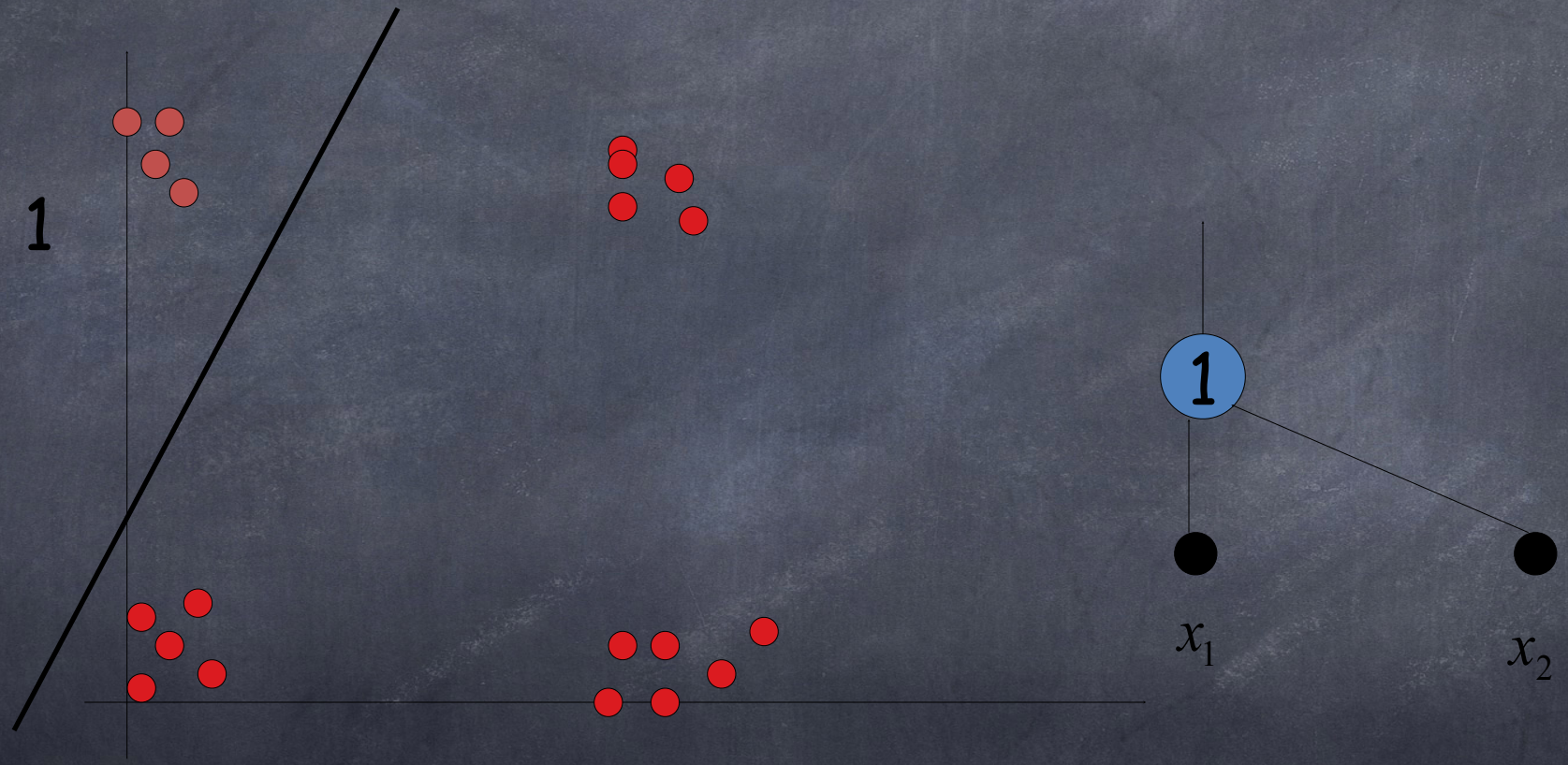
XOR problem



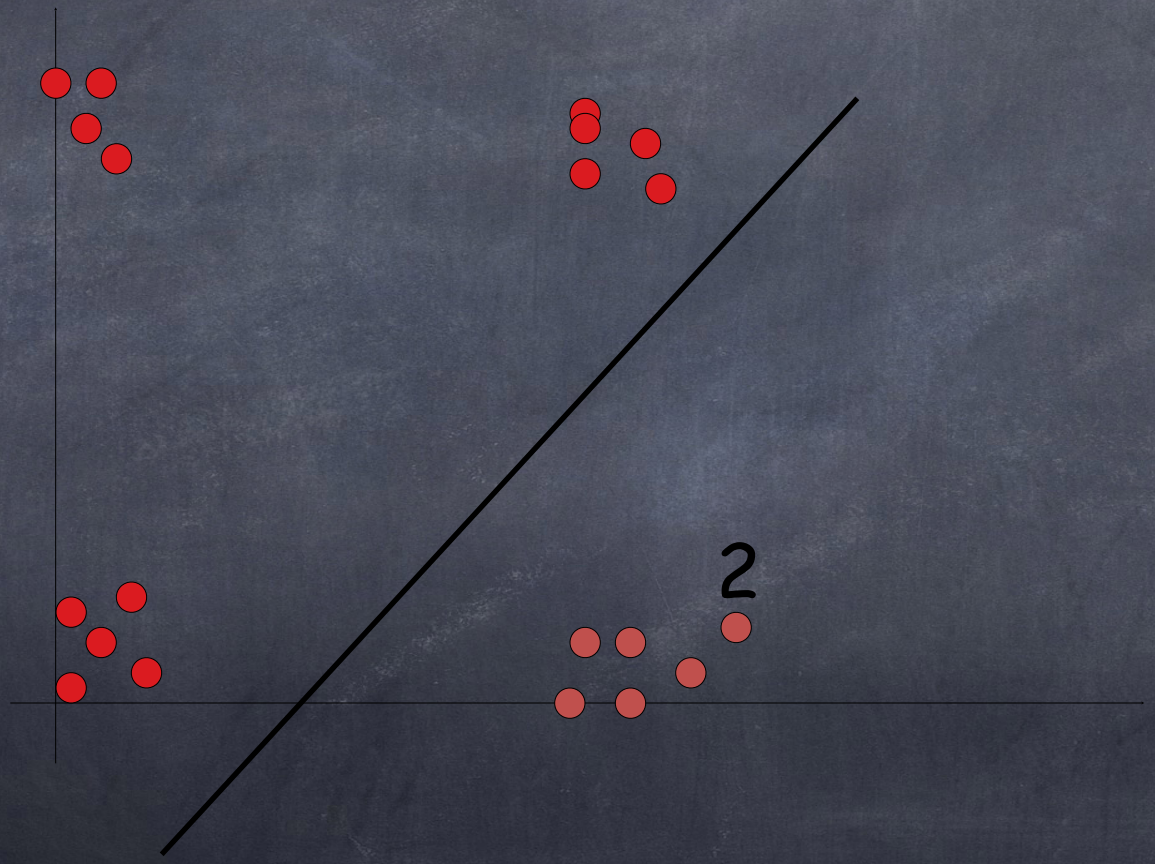
XOR problem

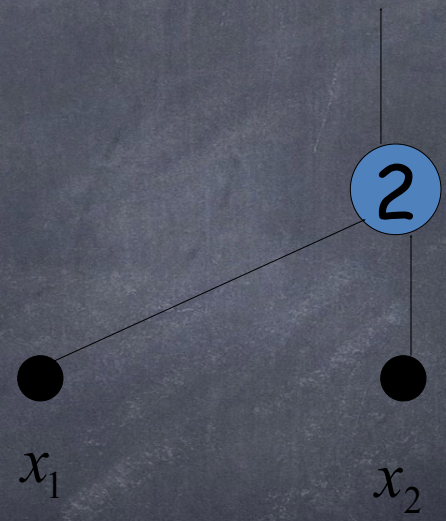


XOR problem

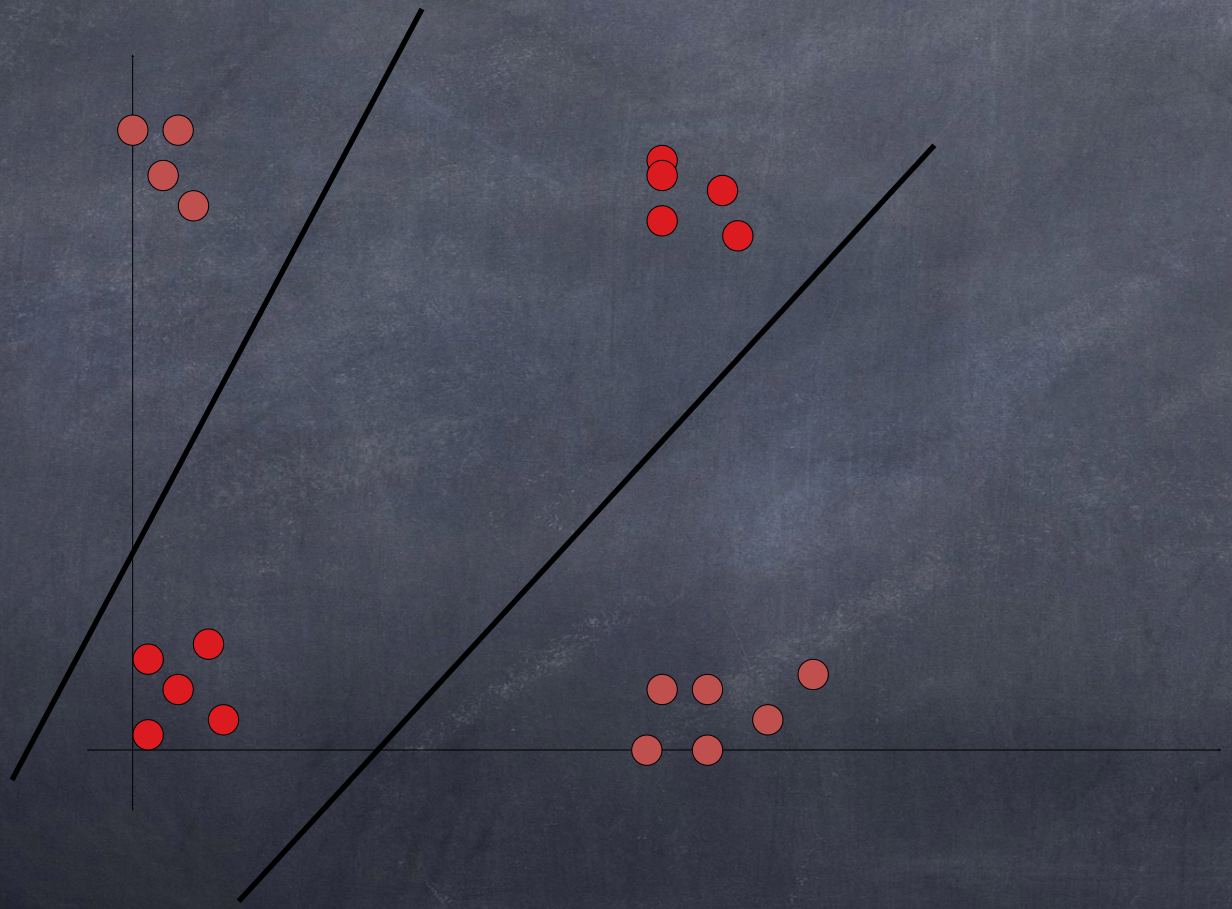


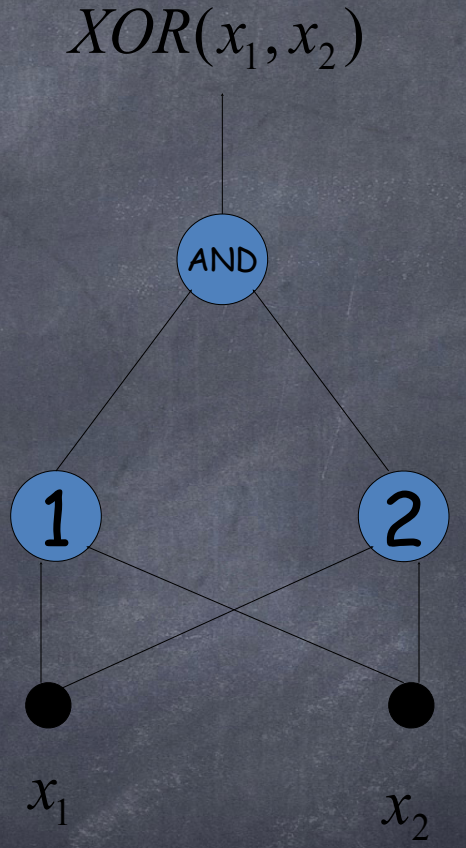
XOR problem





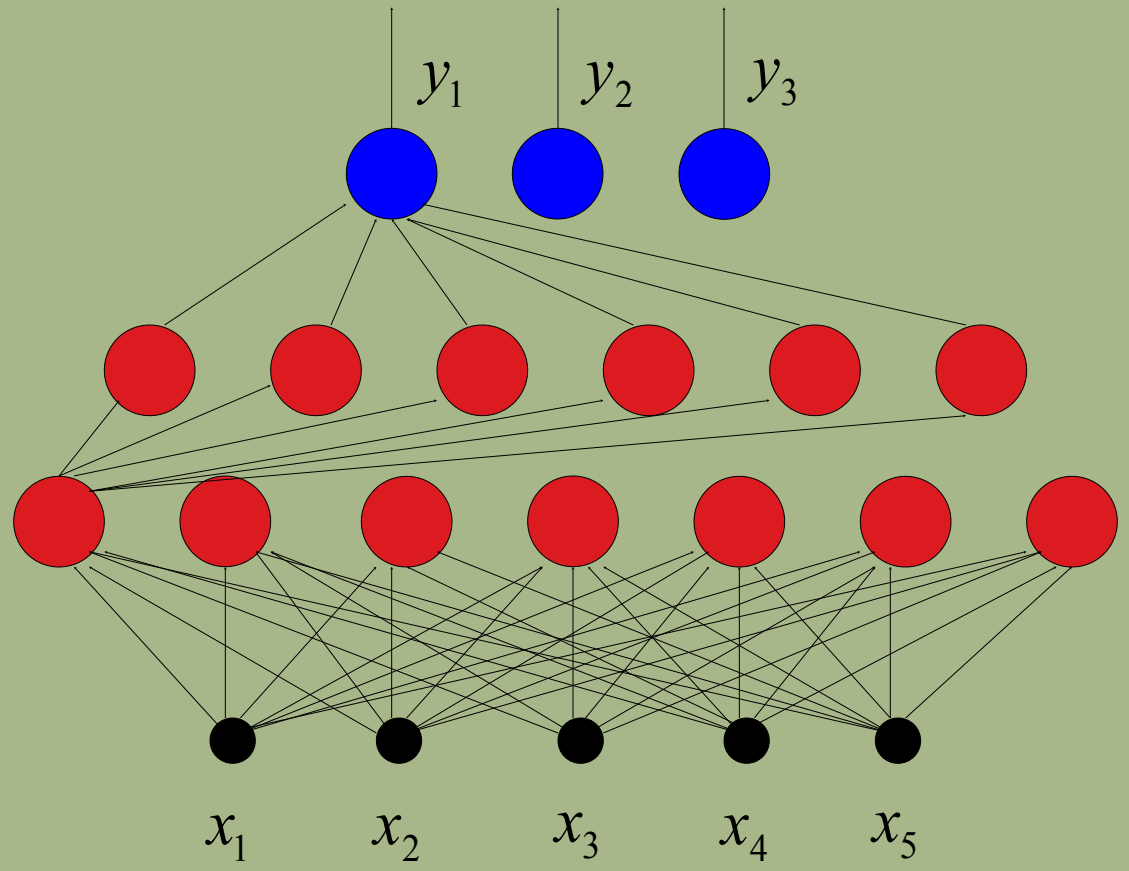
XOR problem





Feed-forward layered network

$$NN : X \rightarrow Y$$



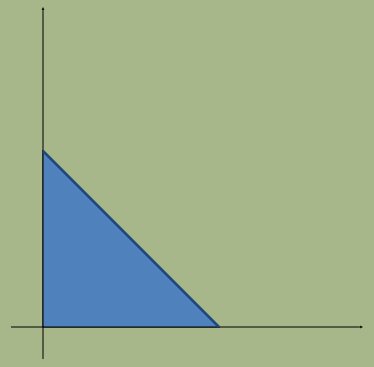
Output layer

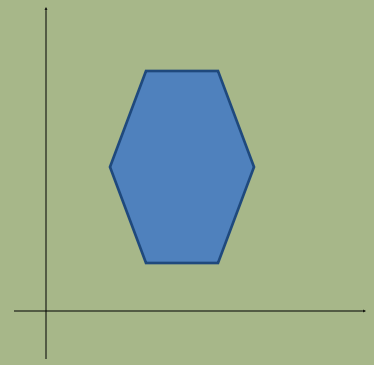
2nd hidden layer

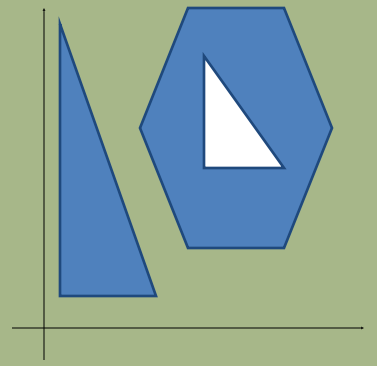
1st hidden layer

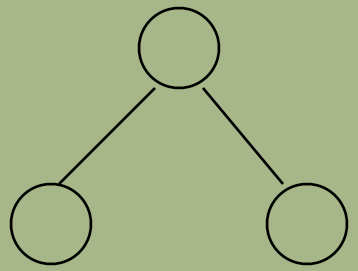
Input layer

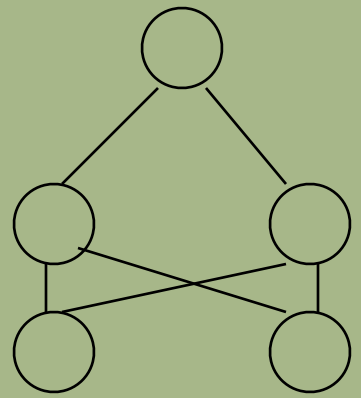
What do each of the layers do?

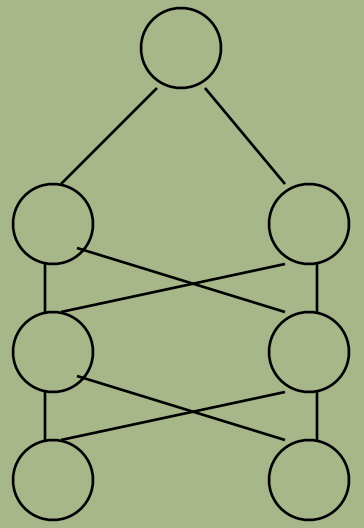










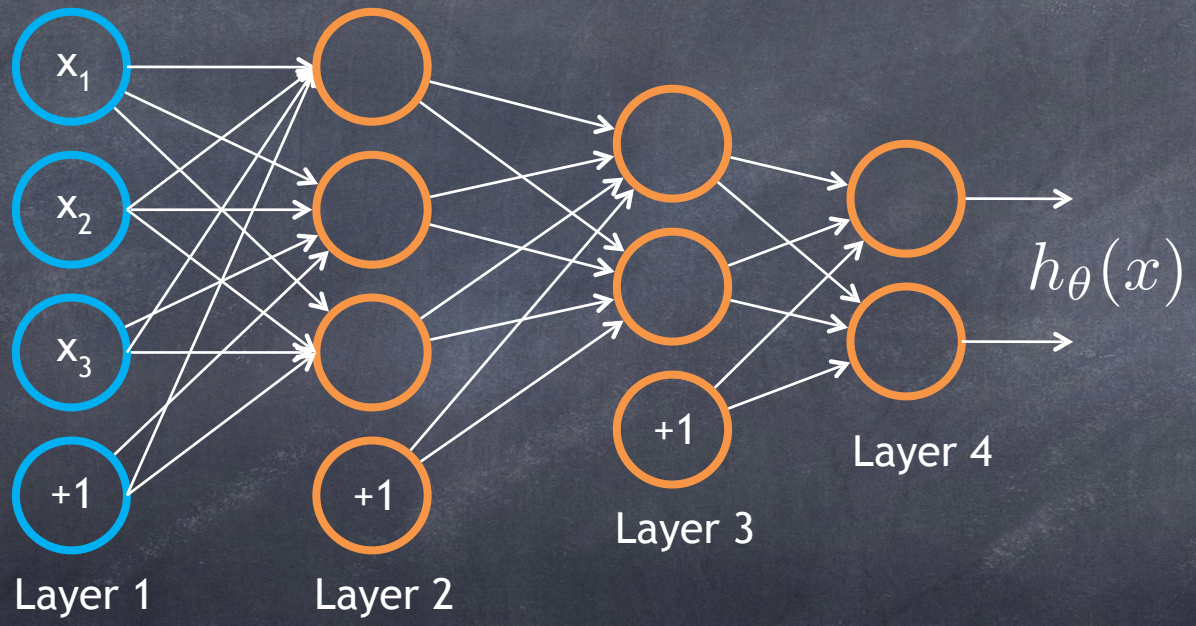


1st layer draws linear boundaries

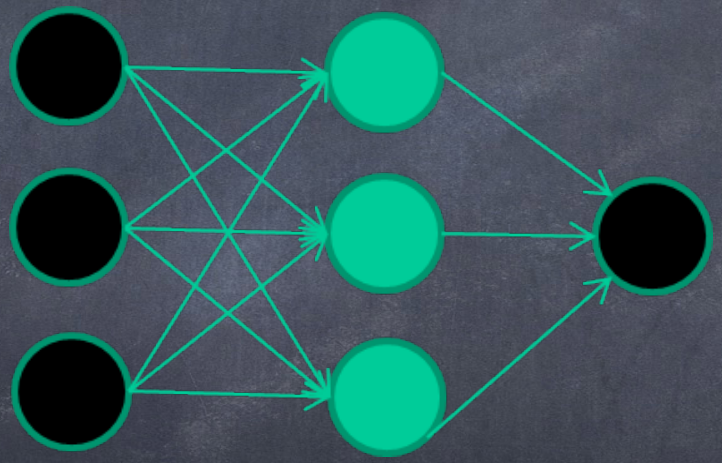
2nd layer combines the boundaries

3rd layer can generate arbitrarily complex boundaries

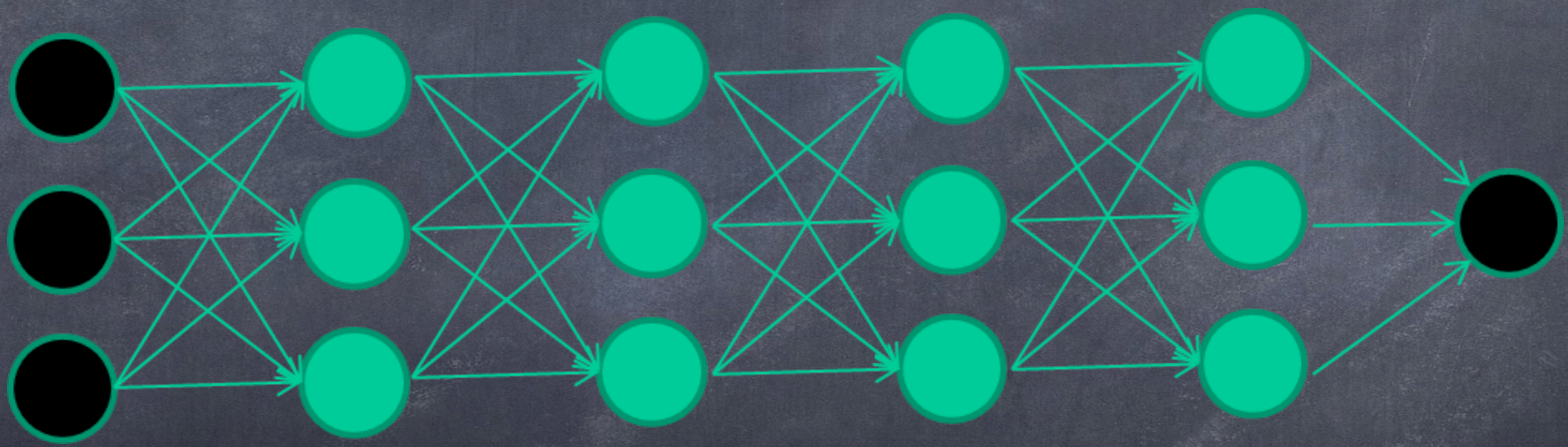
Neural Network



From Single Hidden Layer to Multiple Layers

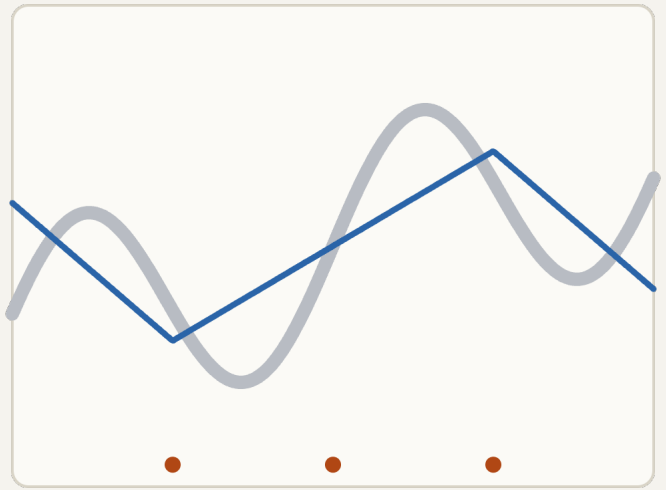


From Single Hidden Layer to Multiple Layers

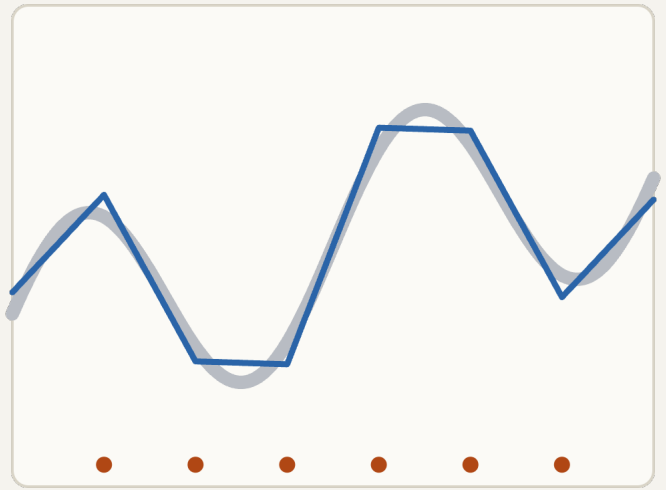


A wide hidden layer can approximate continuous functions

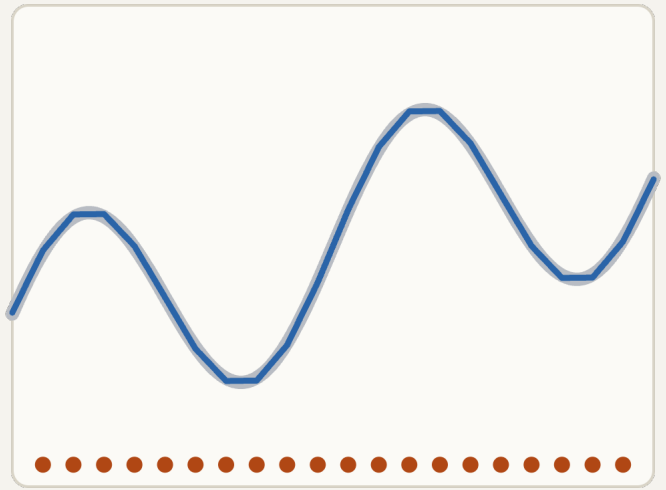
3 hidden ReLU units



6 hidden ReLU units



20 hidden ReLU units



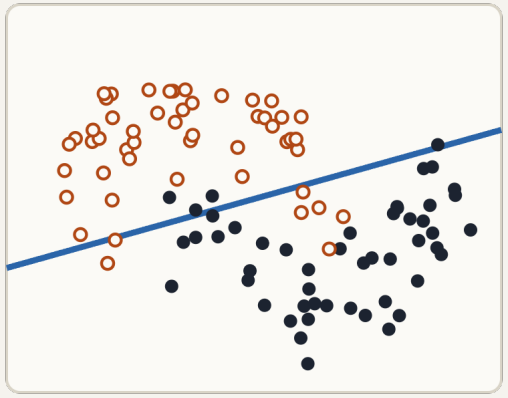
$$f(x) \approx a_0 + a_1 x + \Sigma_k\, a_k\, ReLU(x - c_k)$$

Grey: target function. Blue: one hidden layer of ReLUs. Orange dots: the kinks c_k, one per unit. More units, more kinks, closer fit.

THE MODEL

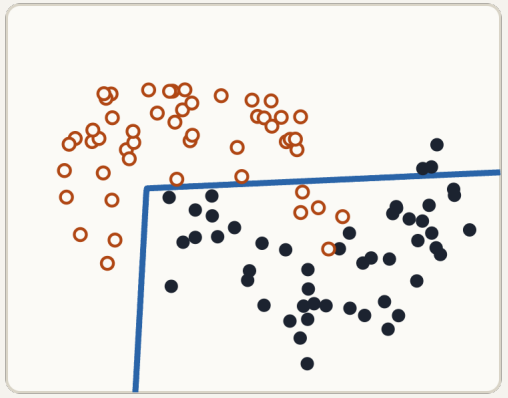
Hidden layers can learn curved boundaries

1 hidden unit



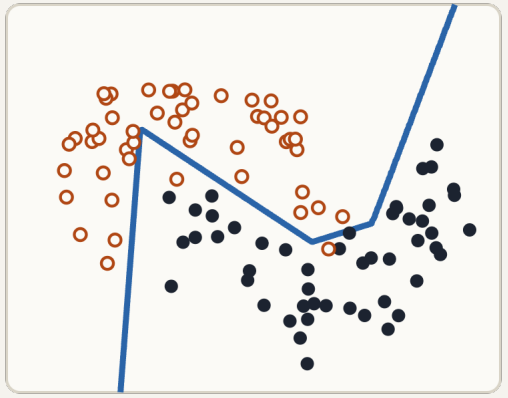
Test accuracy **86.0%**

3 hidden units



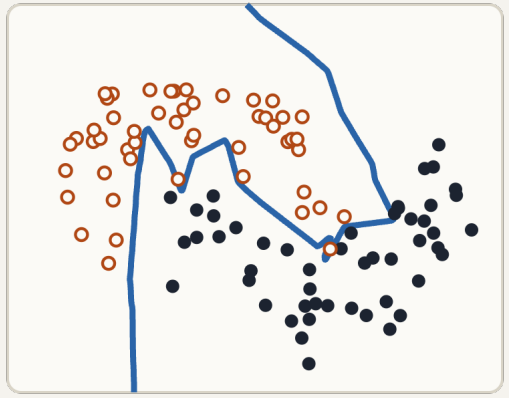
Test accuracy **88.3%**

8 hidden units



Test accuracy **93.4%**

64 hidden units



Test accuracy **93.8%**

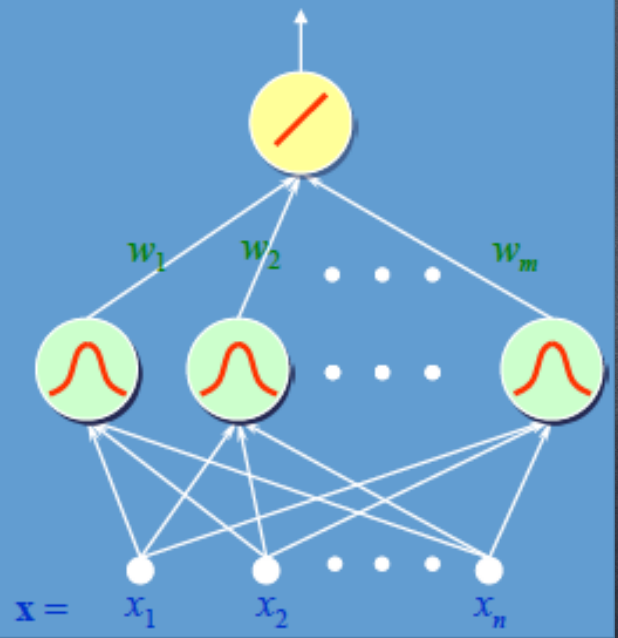
In this illustrative run, logistic regression reaches 86.2%. Multiple hidden units learn nonlinear features. More units do not guarantee better test accuracy.

Training Neural Network

Training set $\mathcal{T} = \left\{ \left(\mathbf{x}^{(k)}, y^{(k)} \right) \right\}_{k=1}^{p}$

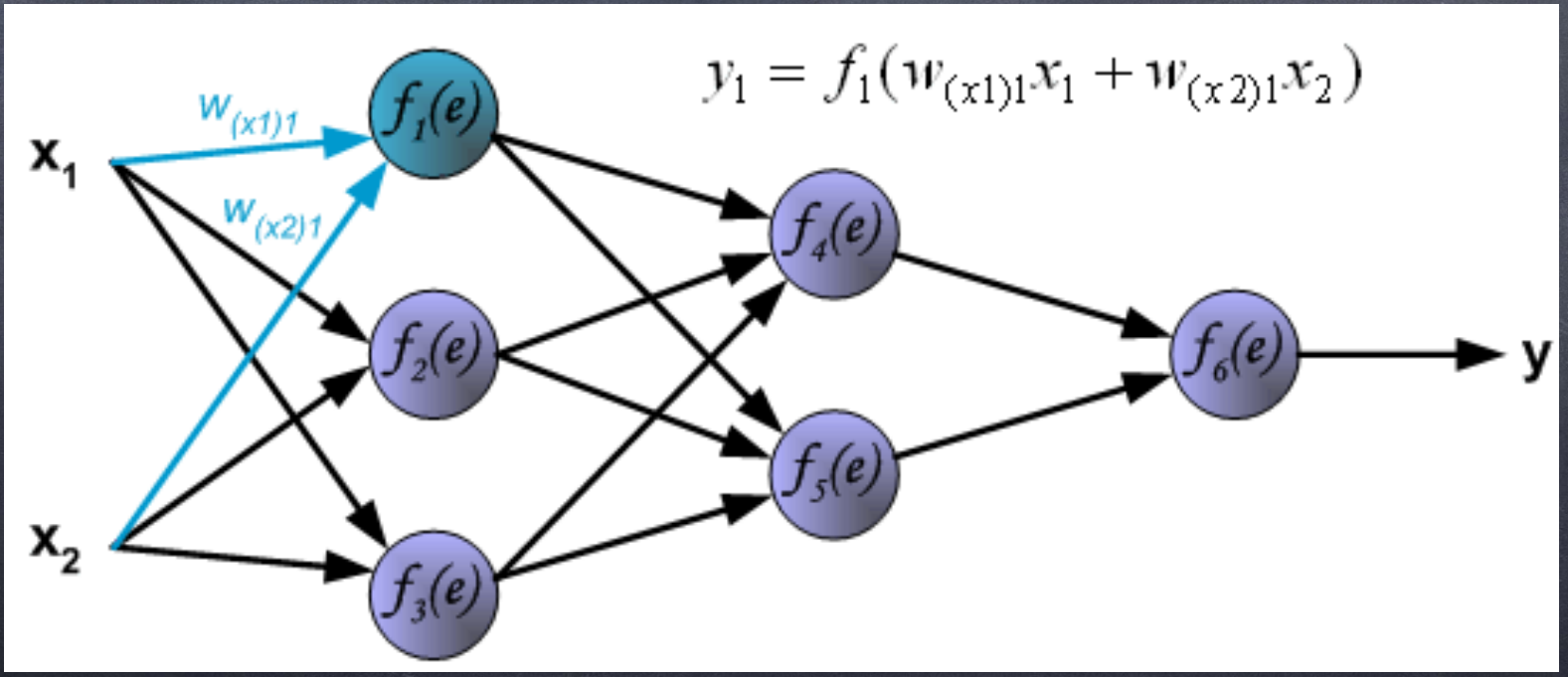
Goal $y^{(k)} \approx f\left(\mathbf{x}^{(k)} \right)$ for all k

$$\min E = \frac{1}{2} \sum_{k=1}^{p} \left[y^{(k)} - f\left(\mathbf{x}^{(k)} \right) \right]^2$$

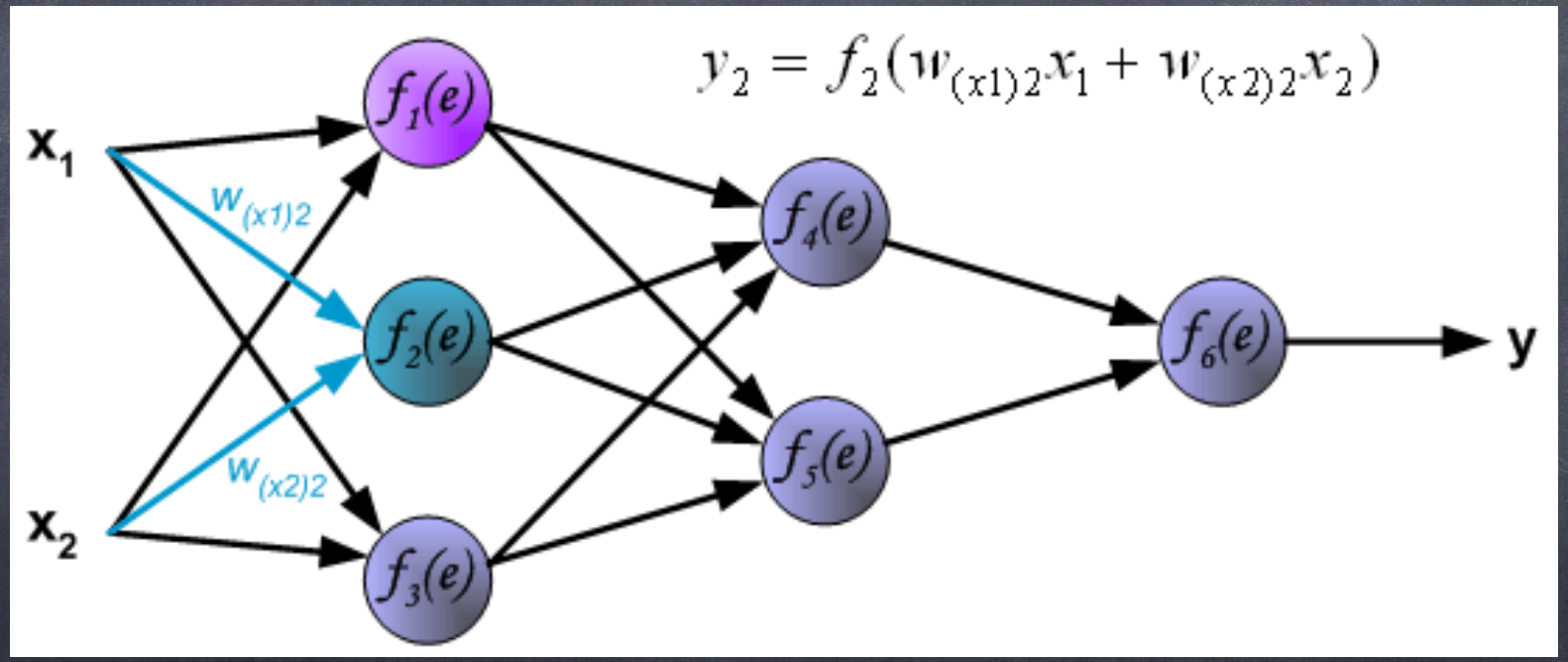


How to Train Neural Network

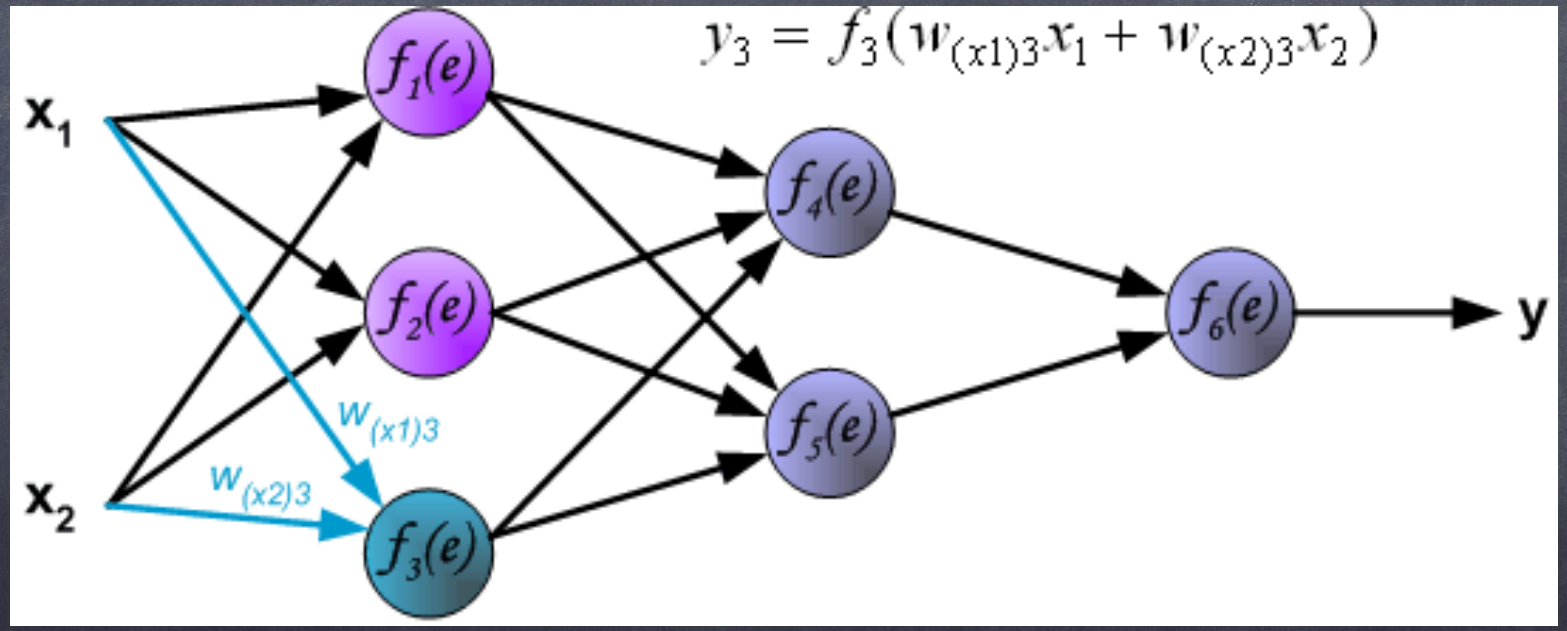
- Back-propagation approach



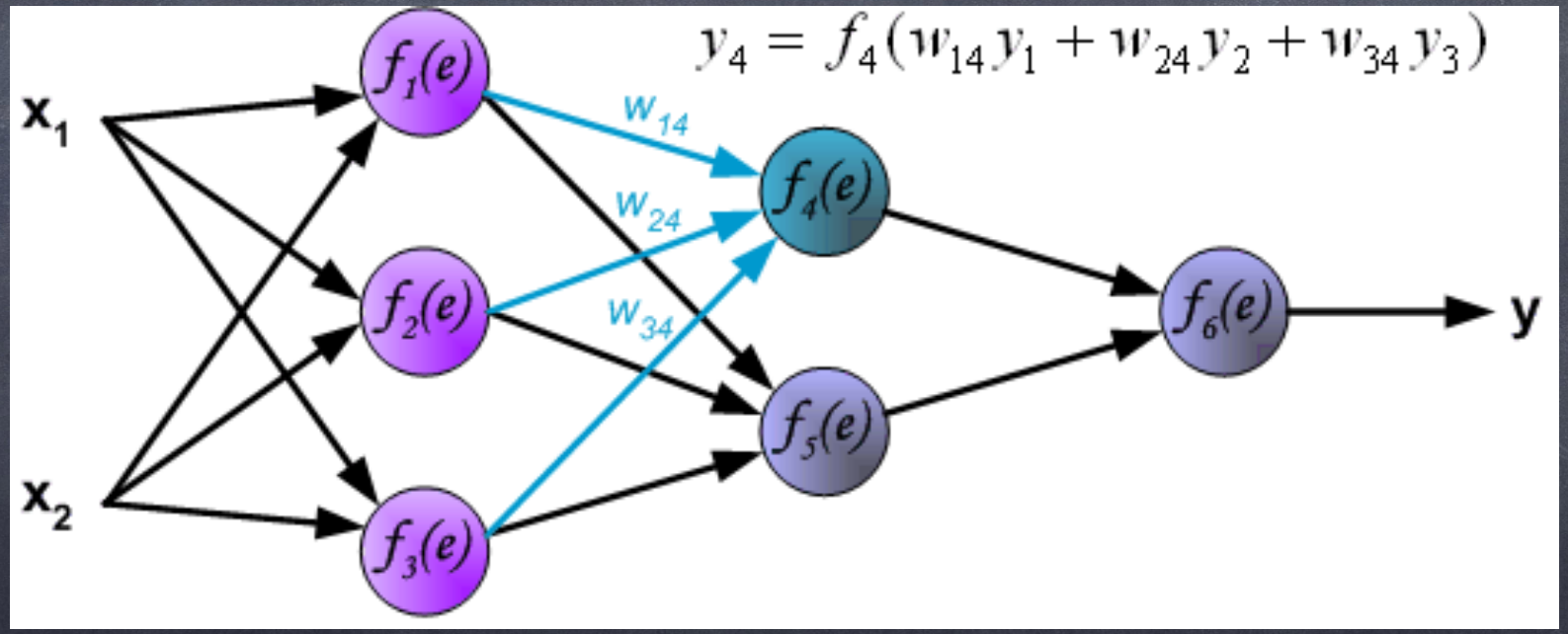
How to Train Neural Network



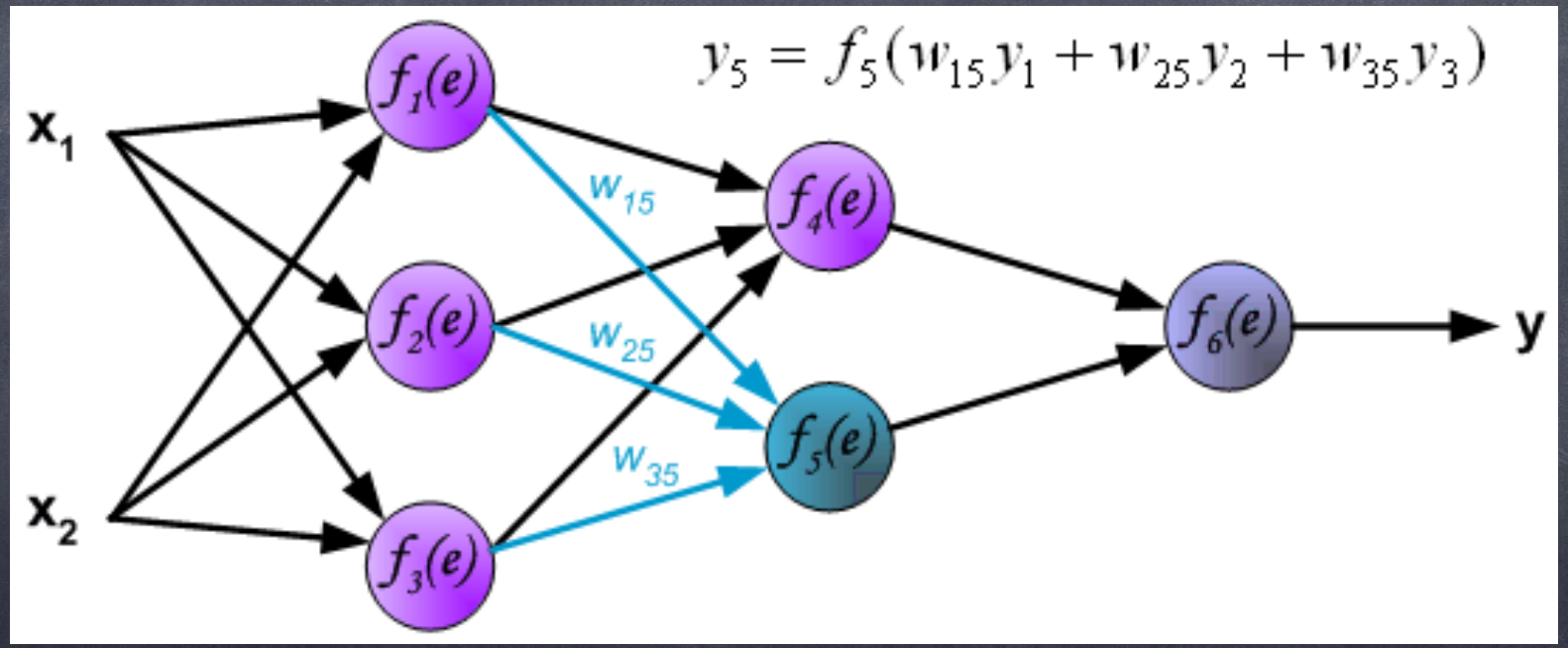
How to Train Neural Network



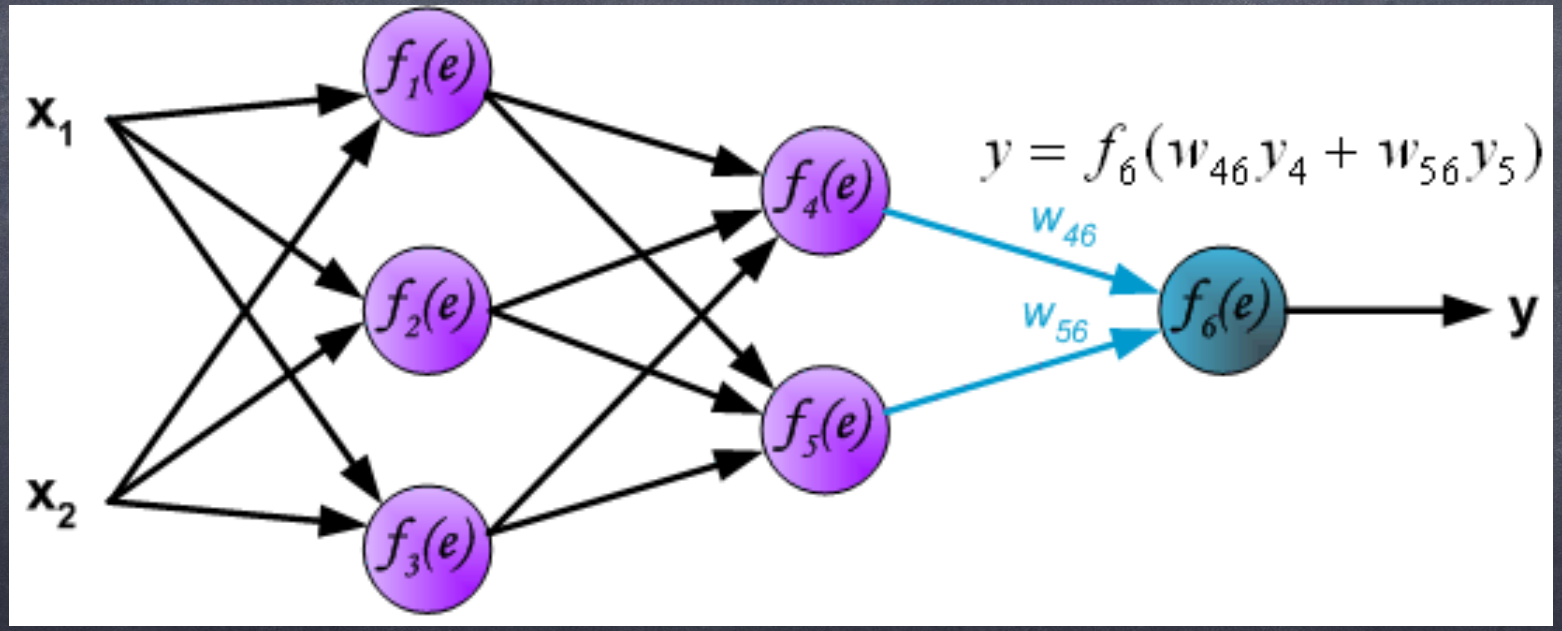
How to Train Neural Network



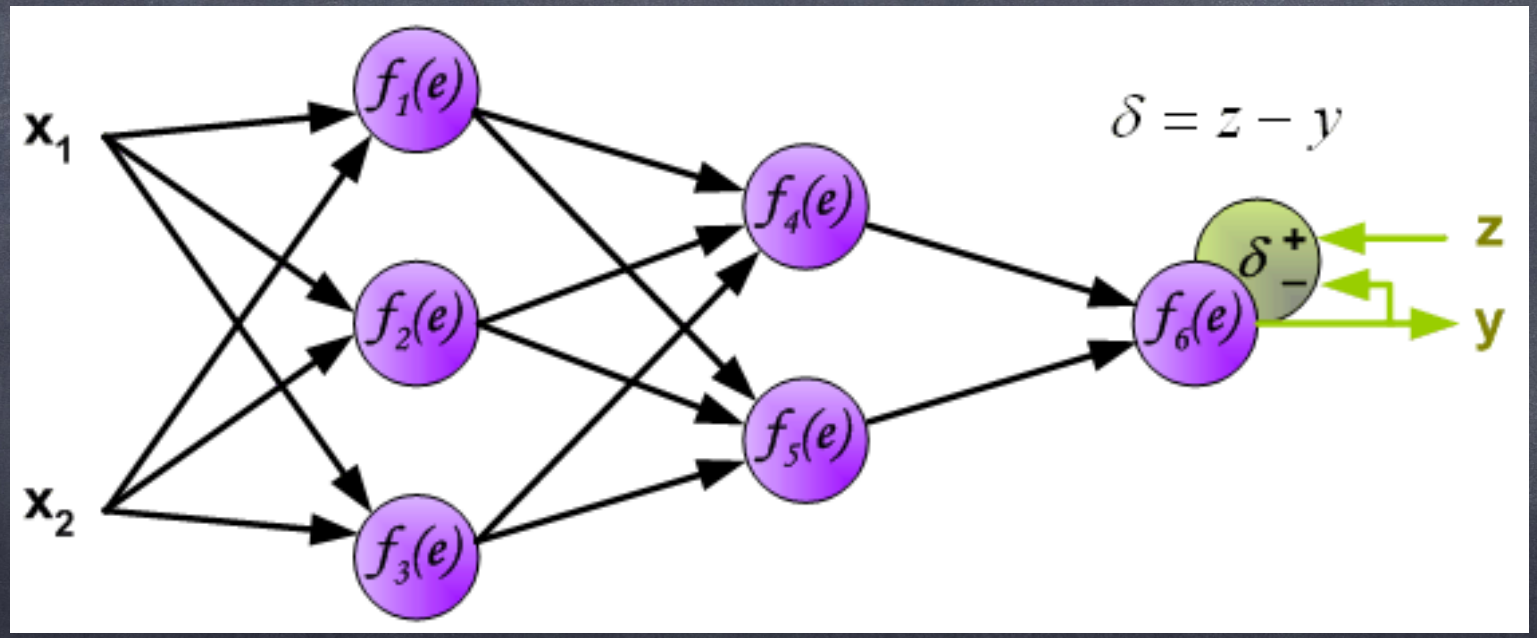
How to Train Neural Network



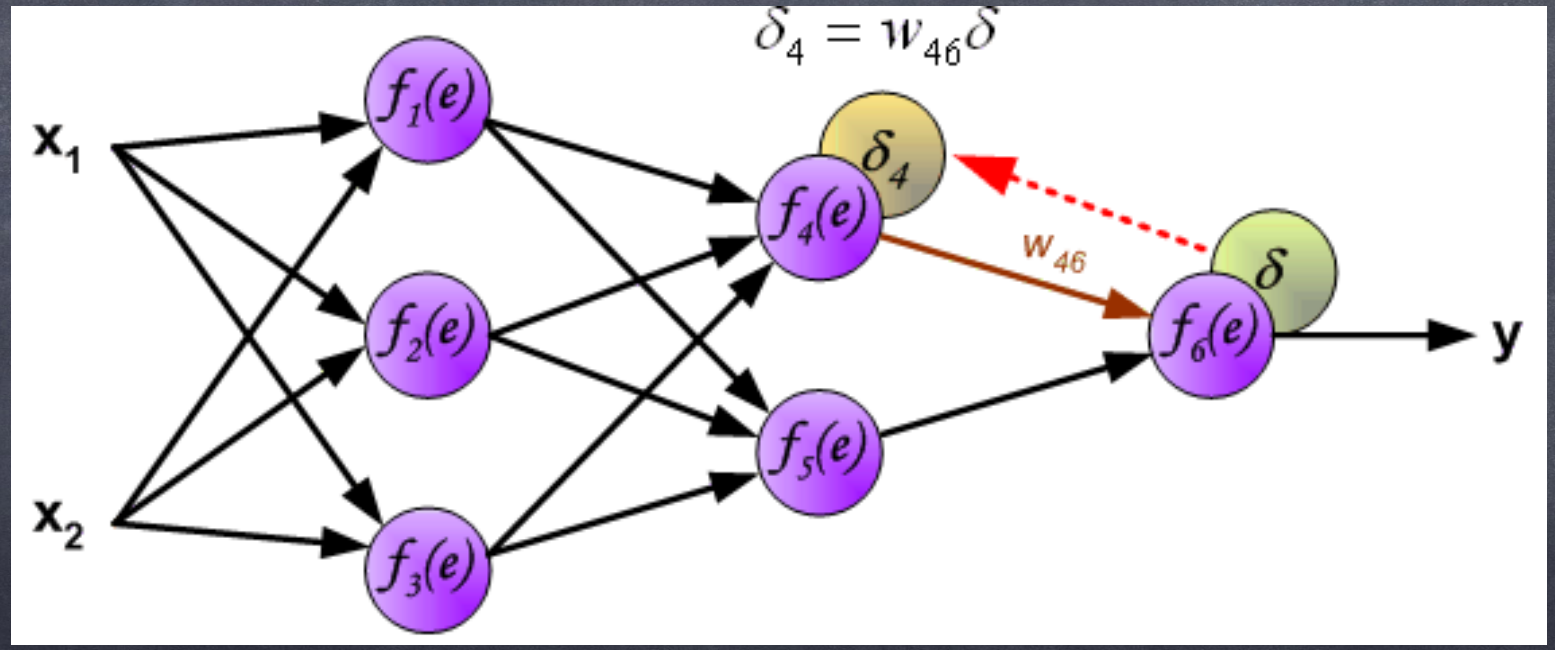
How to Train Neural Network



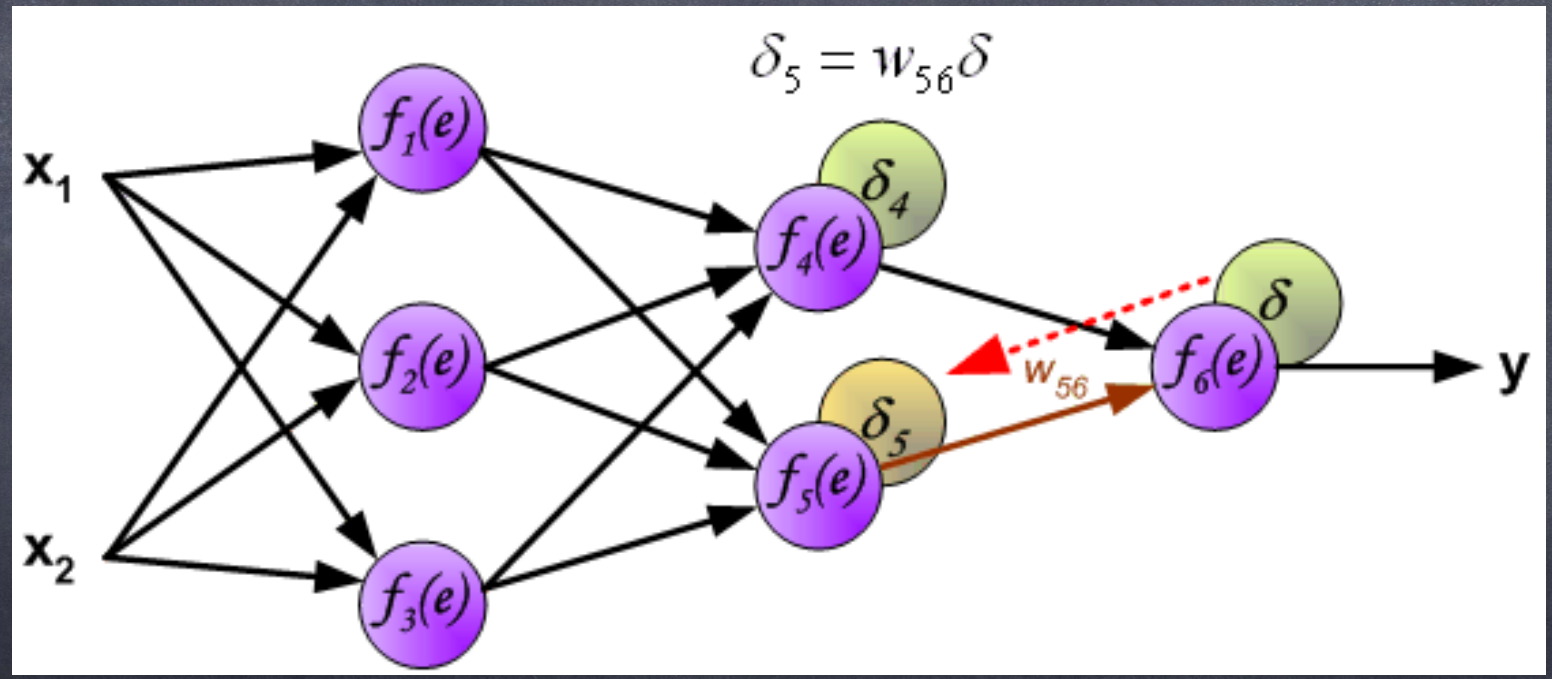
How to Train Neural Network



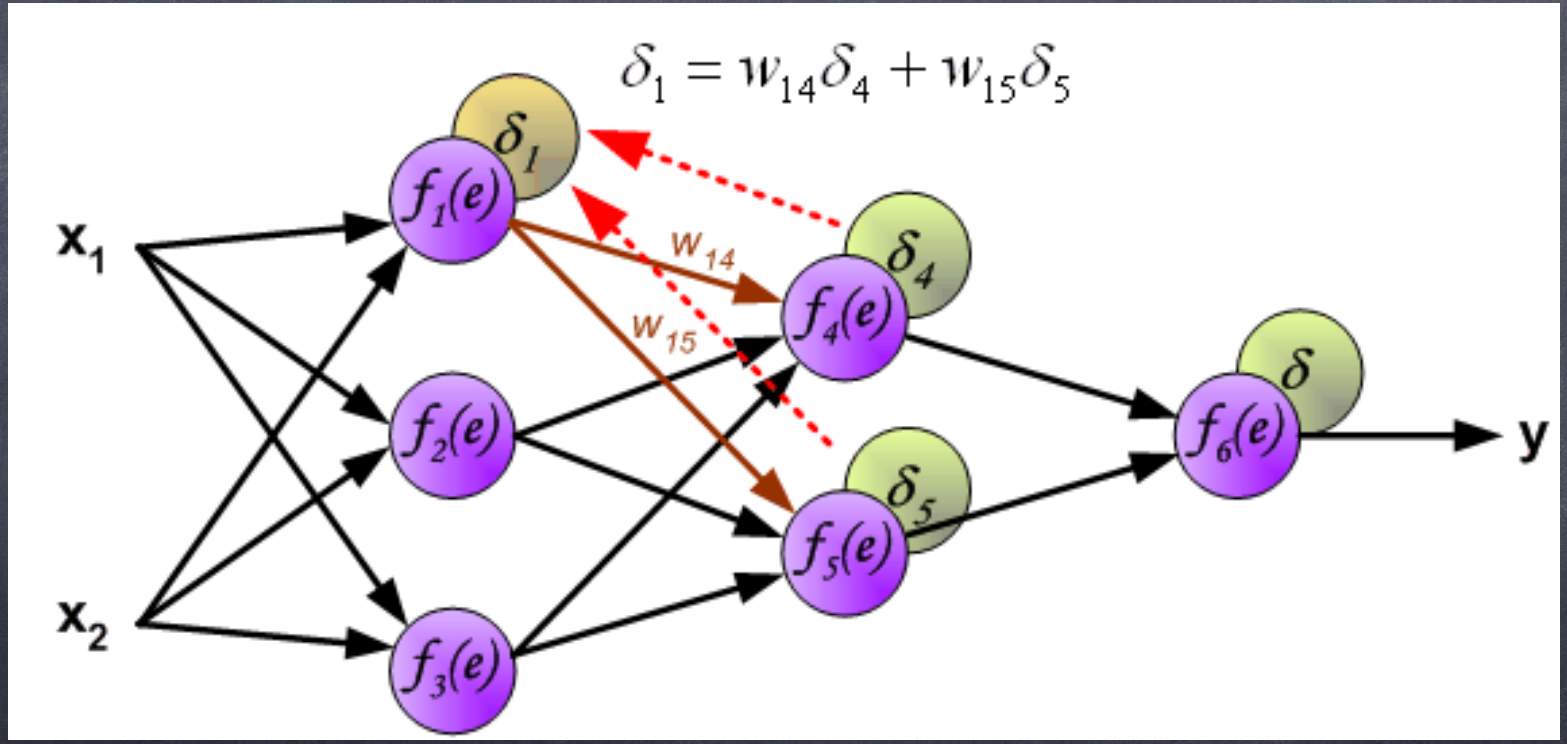
How to Train Neural Network



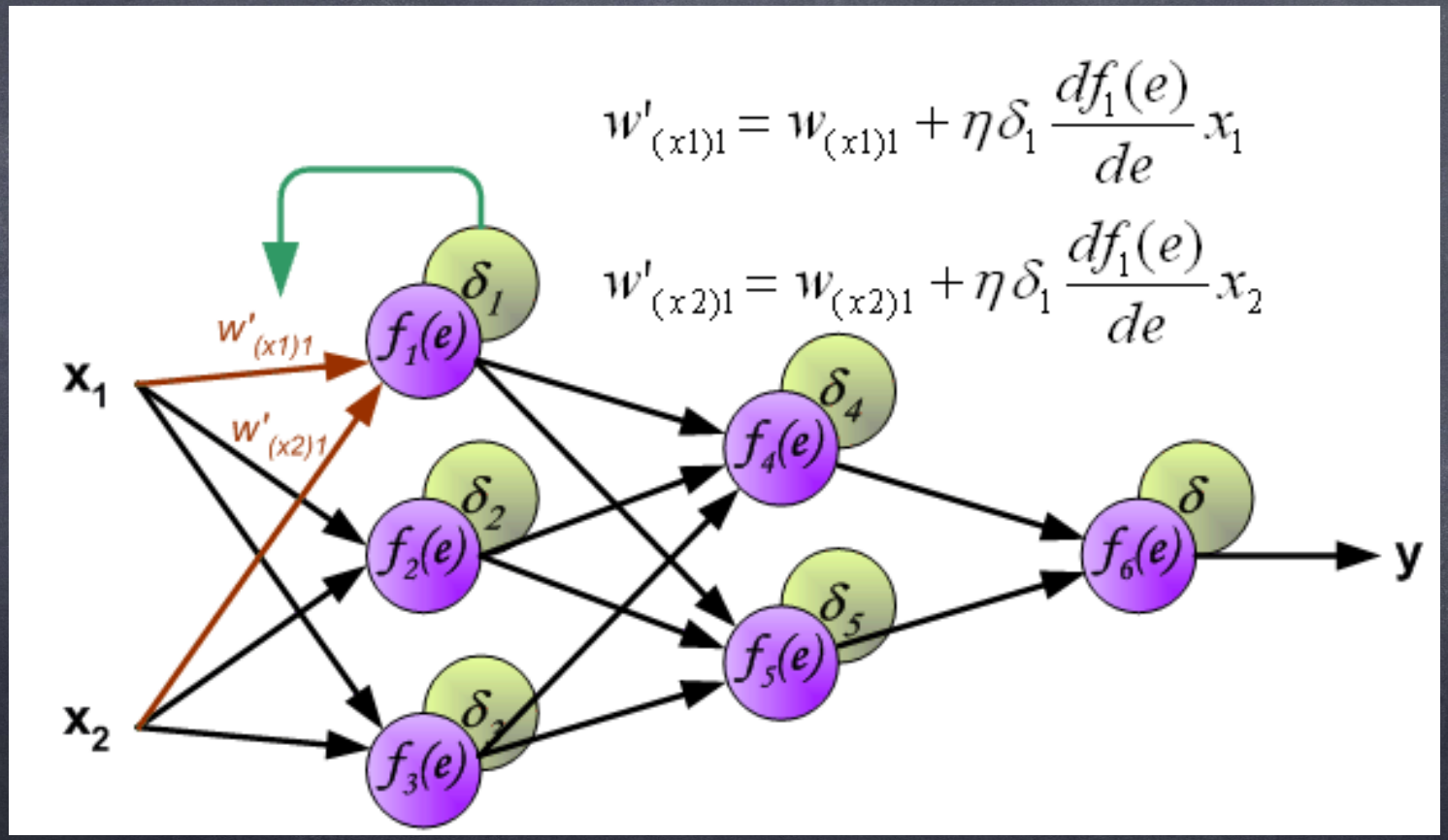
How to Train Neural Network



How to Train Neural Network



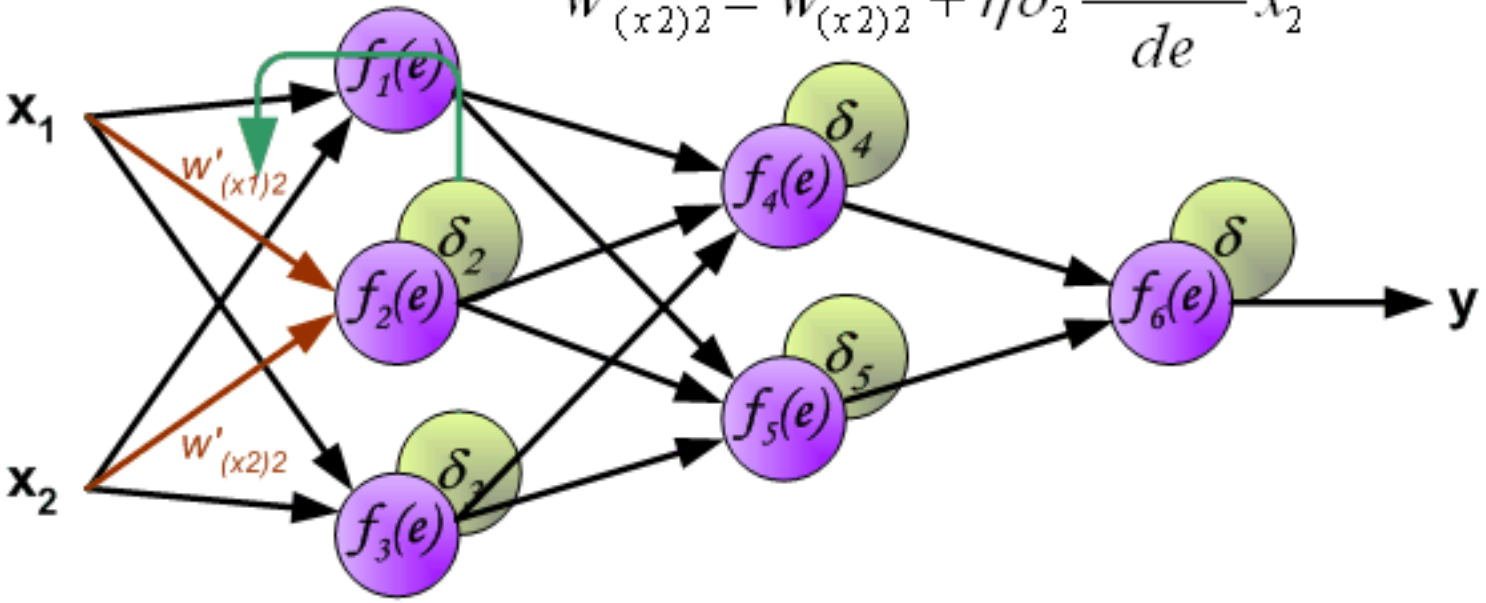
How to Train Neural Network



How to Train Neural Network

$$w'_{(x1)2} = w_{(x1)2} + \eta \delta_2 \frac{df_2(e)}{de} x_1$$

$$w'_{(x2)2} = w_{(x2)2} + \eta \delta_2 \frac{df_2(e)}{de} x_2$$



How to Train Neural Network

